GraphCortex — Complete Study Guide

How to Use This Study Material

This guide is designed to give you total mastery over every line of code in GraphCortex. Read each module in order. By the end, you will be able to explain the entire system to any client, interviewer, or technical stakeholder with full confidence.

Table of Contents

#	Module	File	Phase
01	Project Architecture & Why It Exists	—	Overview
02	Docker & Environment Configuration	`docker-compose.yml`, `.env`	Phase 1
03	Neo4j Connection (Singleton Pattern)	`neo4j_connection.py`	Phase 1
04	Schema Migrations & Vector Indexes	`schema_migrations.py`	Phase 1 + 3
05	Working Memory	`working.py`	Phase 1
06	Episodic Memory	`episodic.py`	Phase 1
07	Semantic Memory & Embeddings	`semantic.py`	Phase 1 + 3
08	Memory Manager Orchestrator	`manager.py`	Phase 1
09	Property Sharding	`sharding.py`	Phase 1

#	Module	File	Phase
10	Retrieval Queries (Cypher)	`retrieval_queries.py`	Phase 2 + 3
11	Lateral Inhibition (The Math)	`inhibition.py`	Phase 2
12	Retrieval Engine (The Brain)	`engine.py`	Phase 2 + 3
13	Logger Configuration	`logger.py`	Phase 3
14	CLI Entry Point	`main.py`	All
15	End-to-End Data Flow Walkthrough	—	All

The One-Paragraph Elevator Pitch

> *GraphCortex is a neuro-symbolic memory and context layer for AI agents. Unlike flat vector databases (Pinecone, Qdrant) that retrieve isolated chunks, GraphCortex stores knowledge in a multi-layered Neo4j Knowledge Graph — tracking real-time conversations (Working Memory), compressing them into chronological events (Episodic Memory), and extracting structured facts (Semantic Memory). When the AI needs to recall information, a brain-inspired Spreading Activation algorithm fans outward from anchor nodes, mathematically bounded by Lateral Inhibition to prevent generic "hub" concepts from flooding the context window. The system uses a hybrid Lexical + Semantic Vector dual-trigger to find anchors, meaning it understands meaning (synonyms), not just exact strings.*

The Folder Structure (Clean Architecture)

```
GraphCortex/
├── src/graph_cortex/
│   ├── core/                          ← THE BRAIN (Pure logic, zero DB deps
│   │   ├── memory/
│   │   │   ├── working.py             ← Real-time conversation buffer
│   │   │   ├── episodic.py            ← Chronological event chains
│   │   │   ├── semantic.py            ← Entity/Concept extraction + embeddi
│   │   │   └── manager.py             ← Orchestrator API
│   │   └── retrieval/
│   │       ├── engine.py              ← Spreading Activation controller
│   │       └── inhibition.py          ← Energy decay math
│   │
│   ├── infrastructure/                ← THE TOOLS (DB-specific, swappable)
│   │   ├── db/
│   │   │   ├── neo4j_connection.py    ← Singleton driver
│   │   │   ├── schema_migrations.py   ← Constraints + Vector indexes
│   │   │   └── queries/
│   │   │       └── retrieval_queries.py ← Raw Cypher queries
│   │   └── storage/
│   │       └── sharding.py            ← Property offloading abstraction
│   │
│   ├── interfaces/                    ← THE DELIVERY (Entry points)
│   │   └── cli/
│   │       └── main.py                ← CLI demo & verification
│   │
│   └── config/
│       └── logger.py                  ← Retrieval event logging
│
├── docker-compose.yml                 ← Neo4j container definition
├── .env                               ← Connection credentials
├── pyproject.toml                     ← Project metadata & deps
├── DECISIONS.md                       ← Architectural decision records
└── Logs/                              ← Timestamped retrieval logs
```

Why This Structure Matters

If a client asks: *"What if we want to switch from Neo4j to AWS Neptune?"*

Your answer: *"We only modify the* `infrastructure/` *folder. The* `core/` *contains pure mathematical logic with zero database imports. The retrieval math, memory orchestration, and energy decay formulas remain entirely untouched."*

This is the **Dependency Inversion Principle** — high-level modules (core) do not depend on low-level modules (infrastructure). Both depend on abstractions.

Module 01: Project Architecture — Why GraphCortex Exists

The Problem GraphCortex Solves

Modern AI agents (ChatGPT, Claude, custom LLM apps) suffer from a fundamental limitation: **they have no persistent, structured memory**. Every conversation starts from scratch. Even with RAG (Retrieval-Augmented Generation), the AI retrieves isolated text chunks from a flat vector database — it cannot understand *how facts relate to each other* or *when they happened*.

What a Flat Vector Database Does

```
User asks: "What did I say about Clean Architecture last week?"

Pinecone/Qdrant response:
  → Chunk 1: "Clean Architecture separates concerns..."
  → Chunk 2: "Infrastructure layer handles database..."
  → Chunk 3: "Domain logic should be pure..."

Problem: These are isolated paragraphs. The AI cannot deduce:
  - WHEN the user said this (chronology)
  - WHO said what (user vs agent)
  - HOW "Clean Architecture" connects to "Neo4j Driver" (topology)
```

What GraphCortex Does

```
User asks: "What did I say about Clean Architecture last week?"

GraphCortex response:
```

```
  → Anchor: "Clean Architecture" (Entity node)
      ├── IS_A_PATTERN_OF → "Software Design" (Concept)
      ├── EXTRACTED_FROM → Event_001 (April 14, "User asked about decoupling
      │   └── SUMMARIZES → Interaction_session_abc
      │       ├── CONTAINS → Message: "How does clean architecture improve..
      │       └── CONTAINS → Message: "By strictly decoupling the Neo4j driv
      └── Connected via BFS → "Neo4j Driver" → "Infrastructure"

The AI now knows: WHO said WHAT, WHEN, and HOW concepts connect.
```

The Three Memory Layers (Multi-Layer Memory Framework)

GraphCortex models memory the same way cognitive neuroscience models the human brain:

Layer 1: Working Memory (Short-Term Buffer)

Biological analogy: Your brain's "scratchpad" — holds the last 7±2 items you're actively thinking about.

In GraphCortex: Stores the raw, real-time conversation. Each conversation session is an `Interaction` node, and each message (user or agent) is a `Message` node linked chronologically via `[:NEXT]` relationships.

```
(Interaction: session_abc)
    ├── [:CONTAINS] → (Message: "How does clean architecture...")
    │                     └── [:NEXT] → (Message: "By strictly decoupling...
```

Layer 2: Episodic Memory (Chronological Events)

Biological analogy: Your ability to remember *events* — "Last Tuesday, I had a meeting about X."

In GraphCortex: When a conversation ends, it gets compressed into an `Event` node with a human-readable summary. Events are chained chronologically via `[:FOLLOWS]` relationships, creating a strict timeline.

```
(Event_001: "User asked about Clean Architecture")
    ├── [:SUMMARIZES] → (Interaction: session_abc)
    └── [:FOLLOWS] → (Event_002: "User discussed Neo4j indexes")
```

Layer 3: Semantic Memory (Facts & Knowledge)

Biological analogy: Your general knowledge — "Paris is the capital of France" — detached from when/where you learned it.

In GraphCortex: Structural facts are extracted from events. `Entity` nodes (specific things) and `Concept` nodes (abstract ideas) are linked via domain-specific relationships, and traced back to the events they were extracted from.

```
(Entity: "Clean Architecture") --[:IS_A_PATTERN_OF]--> (Concept: "Software D
    └── [:EXTRACTED_FROM] → (Event_001)
```

Clean Architecture: The Three Tiers

Why Not Just Write One Script?

A monolithic script mixing Neo4j queries, LLM calls, and math formulas creates "spaghetti code" that is: - **Un-testable:** You can't test the energy decay math without spinning up a database. - **Un-swappable:** Migrating from Neo4j to Neptune means rewriting everything. - **Un-maintainable:** A change in one place breaks something unrelated.

The Solution: Domain-Driven Design

```
┌──────────────────────────────────────────────────────────┐
│                      INTERFACES                          │
│  (How the outside world talks to us: CLI, REST API)      │
│                                                          │
│  main.py — Entry point, wires everything together        │
└──────────────────────────┬───────────────────────────────┘
                           │ calls
┌──────────────────────────▼───────────────────────────────┐
│                         CORE                             │
│  (Pure logic. ZERO database knowledge.)                  │
│                                                          │
│  memory/manager.py    — Orchestrator                     │
│  memory/working.py    — Working Memory logic             │
│  memory/episodic.py   — Episodic Memory logic            │
│  memory/semantic.py   — Semantic Memory logic            │
│  retrieval/engine.py  — Spreading Activation             │
│  retrieval/inhibition.py — Energy Decay math             │
└──────────────────────────┬───────────────────────────────┘
                           │ calls
┌──────────────────────────▼───────────────────────────────┐
│                   INFRASTRUCTURE                         │
│  (Physical tools. Swappable.)                            │
│                                                          │
│  db/neo4j_connection.py  — Driver singleton              │
│  db/schema_migrations.py — Constraints & indexes         │
│  db/queries/retrieval_queries.py — Raw Cypher            │
│  storage/sharding.py — Property offloading               │
└──────────────────────────────────────────────────────────┘
```

The Key Rule

Dependencies flow DOWNWARD only. The `core` never imports from `interfaces`. The `interfaces` never import directly from `infrastructure`. This means:

- To test the energy decay formula → No database needed.
- To switch from Neo4j to Neptune → Only modify `infrastructure/`.
- To add a REST API → Only add to `interfaces/`.

Why Neo4j Instead of Pinecone/Qdrant?

Feature	Flat Vector DB	Neo4j Graph DB
Nearest-neighbor search	✅ Native	✅ Via Vector Indexes
Multi-hop traversal	❌ Manual intersection	✅ Native BFS/DFS
Chronological chains	❌ Must simulate	✅ `[:FOLLOWS]` relationships
Relationship types	❌ Metadata only	✅ First-class `[:IS_A]`, `[:BELONGS_TO]`
Associative recall	❌ Isolated chunks	✅ Spreading Activation
Topological context	❌ None	✅ Full graph structure

The one-liner: *"Vector databases are flat. Memory is not flat. Memory has topology — things connect to other things, and those connections matter."*

Module 02: Docker & Environment Configuration

Files Covered

- `docker-compose.yml`
- `.env`
- `.env.example`

---

docker-compose.yml — Line by Line

```
version: '3.8'
```

What it does: Declares the Docker Compose file format version. Version 3.8 is the most widely supported modern format.

```
services:
  neo4j:
    image: neo4j:5.18.1
```

What it does: Defines a service named `neo4j` that runs the official Neo4j Docker image, pinned to version `5.18.1`. Pinning the version (not using `latest`) prevents unexpected breaking changes when rebuilding containers.

Client question: *"Why 5.18.1 specifically?"*
Answer: *"Neo4j 5.x introduced native Vector Index support, which we need for Phase 3*

semantic search. We pin the version for reproducibility."

```
    container_name: neo4j_nsdmg
```

What it does: Gives the container a human-readable name instead of a random hash. Makes `docker logs neo4j_nsdmg` and `docker exec neo4j_nsdmg` much easier.

```
    ports:
      - "7474:7474" # HTTP port for Neo4j Browser
      - "7687:7687" # Bolt port
```

What it does: Maps container ports to your local machine. - **7474** → Neo4j Browser (the web UI at `http://localhost:7474` where you can visualise the graph) - **7687** → Bolt protocol (the binary protocol that the Python `neo4j` driver uses to communicate with the database)

```
    environment:
      - NEO4J_AUTH=${NEO4J_USERNAME}/${NEO4J_PASSWORD}
```

What it does: Sets the Neo4j authentication credentials using variables from your `.env` file. The format is `username/password`. When the container starts for the **first time**, it permanently saves this password into the data volume. This is why changing `.env` alone won't work after the first boot — you need to wipe the data volume.

```
      - NEO4J_apoc_export_file_enabled=true
      - NEO4J_apoc_import_file_enabled=true
      - NEO4J_apoc_import_file_use__neo4j__config=true
```

What it does: Enables APOC (Awesome Procedures On Cypher) file import/export capabilities. APOC is a plugin library that extends Cypher with utility procedures. The double underscores `__` map to dots in Neo4j config (`apoc.import.file.use_neo4j_config`).

```
      - NEO4J_PLUGINS=["apoc", "graph-data-science"]
```

What it does: Automatically installs two critical plugins: - **APOC** — Provides conditional procedures like `apoc.do.when` (now deprecated, we migrated to native Cypher) - **Graph Data Science (GDS)** — Provides graph algorithms (PageRank, community detection, etc.) for future phases

```
    volumes:
      - ./data/neo4j/data:/data
      - ./data/neo4j/logs:/logs
      - ./data/neo4j/import:/var/lib/neo4j/import
      - ./data/neo4j/plugins:/plugins
```

What it does: Persists Neo4j data to your local filesystem. Without volumes, all data would be lost when the container restarts. The `./data/neo4j/data` directory contains the actual database files — **this is why wiping this directory resets the password**.

```
    healthcheck:
      test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://local
      interval: 10s
      timeout: 10s
      retries: 5
```

What it does: Docker periodically checks if Neo4j is healthy by hitting the HTTP endpoint. If it fails 5 times in a row, Docker marks the container as unhealthy. This is useful for orchestration tools (Kubernetes, etc.) that need to know if the service is genuinely ready.

.env — The Connection Credentials

```
NEO4J_URI=neo4j://127.0.0.1:7687
NEO4J_USERNAME=neo4j
```

```
NEO4J_PASSWORD=cortex_secure_graph_99!
```

Key Details

Variable	Value	Explanation
`NEO4J_URI`	`neo4j://127.0.0.1:7687`	The connection URI. `neo4j://` is the routing protocol (supports clustering). `bolt://` is the direct protocol. Both work for single-instance setups. We use `neo4j://` because Neo4j Desktop defaults to it.
`NEO4J_USERNAME`	`neo4j`	The default admin username. Neo4j always creates this user on first boot.
`NEO4J_PASSWORD`	`cortex_secure_graph_99!`	The password you set when creating the database.

Why `.env` exists separately from `docker-compose.yml`

The `.env` file is loaded by **both** Docker Compose (for the container) and the Python application (via `python-dotenv`). This "single source of truth" pattern prevents credential drift — you never have one password in Docker and a different one in Python.

Security Note

The `.env` file is listed in `.gitignore` so credentials are never committed to version control. The `.env.example` file provides a safe template for new developers.

Module 03: Neo4j Connection — The Singleton Pattern

File Covered

`src/graph_cortex/infrastructure/db/neo4j_connection.py`

Full Code with Line-by-Line Explanation

```
import os
from neo4j import GraphDatabase
from dotenv import load_dotenv
```

- `os` — Access environment variables from the system.
- `GraphDatabase` — The official Neo4j Python driver's entry point for creating database connections.
- `load_dotenv` — Reads the `.env` file and injects its key-value pairs into `os.environ` so they behave like real system environment variables.

```
load_dotenv()
```

Critical: This must be called at module import time (top-level), not inside a function. It reads `.env` and makes `NEO4J_URI`, `NEO4J_USERNAME`, and `NEO4J_PASSWORD` available via `os.getenv()`.

The Singleton Pattern

```
class Neo4jConnection:
    _instance = None  # Class-level variable shared across ALL instances
```

What is a Singleton?
A Singleton ensures only **one instance** of a class ever exists. No matter how many times you call `Neo4jConnection()`, you always get back the exact same object.

Why do we need it here?
Creating a Neo4j driver is expensive — it establishes a TCP connection pool, negotiates the Bolt protocol version, and authenticates. If every memory layer (`WorkingMemory`, `EpisodicMemory`, `SemanticMemory`) each created their own driver, you'd have 3+ redundant connection pools fighting over sockets. The Singleton ensures one shared pool.

```
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Neo4jConnection, cls).__new__(cls)
```

`__new__` vs `__init__`:
- `__new__` controls **object creation** (allocating memory). - `__init__` controls **object initialization** (setting attributes).

By overriding `__new__`, we intercept the creation step. If `_instance` is `None` (first call), we create the object normally. On every subsequent call, we skip creation and return the existing instance.

```
            uri = os.getenv("NEO4J_URI", "bolt://localhost:7687")
            username = os.getenv("NEO4J_USERNAME", "neo4j")
            password = os.getenv("NEO4J_PASSWORD", "changeme123")
```

Fallback defaults: If the `.env` file is missing or a variable isn't set, these defaults kick in. The default password `changeme123` is intentionally weak to signal "you forgot to configure this."

```
            try:
                cls._instance.driver = GraphDatabase.driver(uri, auth=(usern
            except Exception as e:
                print(f"Failed to connect to Neo4j: {e}")
                cls._instance.driver = None
```

Error handling: If Neo4j is offline, we don't crash the entire application. Instead, `driver` is set to `None`, and downstream code can check for this. The `GraphDatabase.driver()` call doesn't actually connect yet — it creates a lazy connection pool that connects on first use.

```
        return cls._instance
```

Always returns the same instance. Whether it's the first call or the hundredth, `cls._instance` is the same object.

Session Management

```
# Global connection instance
db_connection = Neo4jConnection()
```

Module-level instantiation. When any file imports from `neo4j_connection`, this line executes, creating (or reusing) the Singleton. Every module that imports `get_session` shares the same underlying driver.

```
def get_session():
    """Returns a Neo4j session"""
    driver = db_connection.get_driver()
    if driver:
        return driver.session()
    raise ConnectionError("Neo4j driver is not initialized.")
```

What is a "session"?

A session is a lightweight, short-lived object that borrows a connection from the driver's pool, executes queries, and returns the connection when done. Sessions are designed to be used with Python's `with` statement:

```
with get_session() as session:
    session.run("MATCH (n) RETURN n")
# Session automatically closes here, connection returns to pool
```

Why `raise ConnectionError`?

If the driver is `None` (Neo4j is offline), we fail loudly rather than returning `None` and causing mysterious `NoneType has no attribute 'run'` errors deep in the call stack.

```
def execute_read_query(query: str, **kwargs):
    """Executes a generic read transaction and returns a list of dictionarie
    with get_session() as session:
        result = session.run(query, **kwargs)
        return [record.data() for record in result]
```

Utility function for simple read-only queries. Converts Neo4j `Record` objects into plain Python dictionaries. The `**kwargs` allows passing Cypher parameters:

```
execute_read_query("MATCH (n:Entity {name: $name}) RETURN n", name="Clean Ar
```

How Other Files Use This

Every memory layer imports the same function:

```
from graph_cortex.infrastructure.db.neo4j_connection import get_session
```

```
with get_session() as session:
    session.run(query, ...)
```

They all share the same Singleton driver → same connection pool → efficient resource usage.

Common Client Questions

Q: "What happens if Neo4j goes down mid-operation?"
A: The Neo4j Python driver has built-in retry logic and connection health checks. If a connection drops, the driver automatically tries to acquire a new one from the pool. For the current implementation, if the initial connection fails entirely, `driver` is `None` and all operations raise `ConnectionError`.

Q: "Is this thread-safe?"
A: Yes. The Neo4j Python driver's connection pool is thread-safe by design. Multiple threads can call `get_session()` concurrently, and each gets its own session from the shared pool.

Q: "Why not use a connection per request?"
A: Creating a new TCP connection for every query is extremely expensive (TCP handshake, TLS negotiation, Bolt protocol negotiation, authentication). Connection pooling amortizes this cost across hundreds of queries.

Module 04: Schema Migrations & Vector Indexes

File Covered

`src/graph_cortex/infrastructure/db/schema_migrations.py`

Full Code with Line-by-Line Explanation

```
from graph_cortex.infrastructure.db.neo4j_connection import get_session
```

Imports our Singleton session factory. Every query in this file runs through the shared connection pool.

```
# Configuration for Semantic Vector Search
# sentence-transformers (BAAI/bge-base-en-v1.5) uses 768-dimensional vectors
# If migrating to OpenAI embeddings later, change this to 1536 and re-initia
VECTOR_DIMENSION = 768
```

Why is this a module-level constant?
The vector dimension is a schema-level decision that affects multiple components: 1. The Neo4j vector index must be created with a specific dimension. 2. The embedding model must output vectors of that exact dimension. 3. Every query vector must also match.

By centralising it here, you change one number to switch models. For example: - `all-MiniLM-L6-v2` → 384 - `BAAI/bge-base-en-v1.5` → 768 - `OpenAI text-embedding-3-small` → 1536

The `initialize_schema()` Function

```
def initialize_schema():
    """
    Initializes constraints and indexes for the Multi-Layer Memory Framework
    """
```

This function runs once when the application starts (called from `main.py`). It's **idempotent** — running it multiple times is safe because every query uses `IF NOT EXISTS`.

Part 1: Base Constraints

```
    base_queries = [
        # Working Memory Constraints
        "CREATE INDEX IF NOT EXISTS FOR (i:Interaction) ON (i.timestamp)",
        "CREATE INDEX IF NOT EXISTS FOR (m:Message) ON (m.message_id)",
```

Indexes speed up lookups. When you `MATCH (i:Interaction {session_id: ...})`, Neo4j can jump directly to the right node instead of scanning every node.

- `Interaction.timestamp` — Indexed so we can quickly sort/filter interactions by time.
- `Message.message_id` — Indexed so we can quickly find specific messages.

```
        # Episodic Memory Constraints
        "CREATE CONSTRAINT IF NOT EXISTS FOR (e:Event) REQUIRE e.event_id IS
        "CREATE INDEX IF NOT EXISTS FOR (e:Event) ON (e.timestamp)",
```

Constraints vs Indexes: - A **constraint** (`REQUIRE ... IS UNIQUE`) enforces data integrity. Neo4j will physically reject any attempt to create two Events with the same `event_id`. It also automatically creates an index. - An **index** only speeds up lookups, but doesn't enforce uniqueness.

We use a constraint on `event_id` because every event must be globally unique (we generate UUIDs).

```
        # Semantic Memory Constraints
        "CREATE CONSTRAINT IF NOT EXISTS FOR (e:Entity) REQUIRE e.name IS UN
        "CREATE CONSTRAINT IF NOT EXISTS FOR (c:Concept) REQUIRE c.name IS U
    ]
```

Entities and Concepts are identified by their `name` (e.g., `"Clean Architecture"`). The uniqueness constraint ensures that `MERGE (e:Entity {name: "Clean Architecture"})` either finds the existing node or creates a new one — never a duplicate.

Part 2: Vector Indexes (Phase 3)

```
    vector_queries = [
        "DROP INDEX entity_vector_index IF EXISTS",
        "DROP INDEX concept_vector_index IF EXISTS",
```

Why DROP before CREATE?

Vector indexes are dimension-locked. If you previously created a 384-dimensional index and now want 768, Neo4j won't let you modify it in-place. You must drop and recreate. `IF EXISTS` prevents errors if the index doesn't exist yet.

```
        f"CREATE VECTOR INDEX entity_vector_index IF NOT EXISTS FOR (e:Entit
        f"CREATE VECTOR INDEX concept_vector_index IF NOT EXISTS FOR (c:Conc
    ]
```

Breaking this Cypher down:

Part	Meaning
`CREATE VECTOR INDEX`	Creates a special index type for high-dimensional vectors

Part	Meaning
`entity_vector_index`	Human-readable name we reference later in queries
`FOR (e:Entity)`	Applies to all nodes with the `Entity` label
`ON (e.embedding)`	Indexes the `embedding` property (the 768-dim float array)
`vector.dimensions: 768`	The index expects exactly 768-dimensional vectors
`vector.similarity_function: 'cosine'`	Uses cosine similarity (angle between vectors) for scoring. Range: 0 (orthogonal) to 1 (identical).

Neo4j 5.x note: Older documentation uses `similarity.function` (with a dot). Neo4j 5.x renamed it to `vector.similarity_function`. This was one of the compatibility fixes we made.

Part 3: Execution with Diagnostic Error Handling

```
try:
    with get_session() as session:
        # 1. Base Constraints
        for query in base_queries:
            session.run(query)
        print("[INFO] Core Database Schema initialized successfully.")
```

Runs all base constraint queries sequentially. These always work on any Neo4j 5.x instance.

```
        # 2. Vector Diagnostic Check
        try:
            for v_query in vector_queries:
                session.run(v_query)
            print(f"[INFO] Vector Indexes ({VECTOR_DIMENSION}-Dimensions
        except Exception as v_err:
            print(f"\n[WARNING] Neo4j Vector Initialization Failed.")
```

```
                    print(f"[DIAGNOSTIC] Your current Neo4j Container version ma
                    print(f"[DIAGNOSTIC] Inner Error: {v_err}\n")
```

Nested try/except pattern:
The vector index creation is wrapped in its **own** try/except, separate from the base constraints. This means: - If vector creation fails (e.g., old Neo4j version), the base schema is still created successfully. - The error message specifically diagnoses the vector issue rather than a generic "schema failed." - The application can continue running in "lexical-only" mode without vectors.

```
    except Exception as e:
        print(f"[ERROR] Failed to initialize core schema: {e}")
```

The outer try/except catches connection-level failures (Neo4j offline, wrong password).

How Vector Indexes Work Internally

When you set `e.embedding = [0.123, -0.456, ...]` on a node, Neo4j's vector index uses an **HNSW (Hierarchical Navigable Small World)** algorithm internally. Think of it as a multi-layer skip-list optimised for high-dimensional nearest-neighbor search.

When you later call `db.index.vector.queryNodes('entity_vector_index', 2, $vector)`, Neo4j uses HNSW to find the 2 closest vectors in O(log n) time, avoiding the O(n) brute-force scan.

Common Client Questions

Q: "What happens if we need to change embedding models?"
A: Change `VECTOR_DIMENSION` in this file, change the model string in `semantic.py` and

`engine.py`, then re-run the schema migration. The DROP+CREATE pattern rebuilds the indexes cleanly. You'll also need to re-embed existing nodes (one-time batch job).

Q: "Why cosine similarity and not Euclidean distance?"
A: Cosine similarity measures the *angle* between vectors, not the *magnitude*. This means "Clean Architecture" and "Software Design" will be scored similarly regardless of how long or short the original text was. Euclidean distance is sensitive to vector magnitude, which is undesirable for text embeddings.

Module 05: Working Memory

File Covered

`src/graph_cortex/core/memory/working.py`

What Working Memory Does

Working Memory is the **real-time conversation buffer**. It stores the raw back-and-forth between user and agent as it happens, before any summarisation or knowledge extraction occurs.

Graph structure it creates:

```
(Interaction: session_abc)
    ├── [:CONTAINS] → (Message: role="user", content="How does clean archite
    │                     └── [:NEXT] →
    └── [:CONTAINS] → (Message: role="agent", content="By strictly decouplin
```

Full Code with Line-by-Line Explanation

Imports

```
import uuid
from datetime import datetime
```

```
from graph_cortex.infrastructure.db.neo4j_connection import get_session
```

- `uuid` — Generates globally unique identifiers for each message (e.g., `a1b2c3d4-e5f6-...`).
- `datetime` — Captures the exact timestamp when each node is created.
- `get_session` — Our Singleton Neo4j session factory from Module 03.

Class Definition

```
class WorkingMemory:
    """
    Handles real-time bounded interactions.
    This acts as a short-term buffer before memories are summarized into epi
    or semantic structures.
    """
    def __init__(self):
        pass
```

The `__init__` does nothing because `WorkingMemory` is stateless — it has no instance variables. Every operation goes directly to Neo4j. This is a deliberate Clean Architecture choice: the class is a **service**, not a data container.

Method 1: `add_interaction()`

```
    def add_interaction(self, session_id: str):
        """Creates a new Interaction session node."""
        query = """
        MERGE (i:Interaction {session_id: $session_id})
        ON CREATE SET i.timestamp = $timestamp, i.created_at = $timestamp
        RETURN i
        """
```

Cypher breakdown:

Keyword	What It Does
`MERGE`	"Find or Create." If an Interaction with this `session_id` exists, reuse it. Otherwise, create it. This is **idempotent** — calling it twice with the same ID won't create a duplicate.
`ON CREATE SET`	Only executes the `SET` clause when a **new** node is created (not when an existing one is found). This prevents overwriting the original timestamp on subsequent calls.
`$session_id`	A parameterised variable. Never string-interpolated — this prevents Cypher injection attacks.

```
        with get_session() as session:
            session.run(query, session_id=session_id, timestamp=datetime.now
        return session_id
```

`with get_session() as session` — The context manager ensures the session is properly closed even if an exception occurs. The `session.run()` method sends the Cypher query to Neo4j with the provided parameters.

`datetime.now().isoformat()` — Produces `"2026-04-16T18:47:41.123456"`. ISO 8601 format is universally parseable and sorts lexicographically.

Method 2: `add_message()` — The Complex One

```
    def add_message(self, session_id: str, role: str, content: str):
        """Appends a new message to an interaction session."""
        message_id = str(uuid.uuid4())
```

Every message gets a UUID. This guarantees global uniqueness without a central counter.

```
        query = """
        MATCH (i:Interaction {session_id: $session_id})
```

Step 1: Find the Interaction node for this session. If `add_interaction()` wasn't called first, this will match nothing and the entire query returns `None`.

```
        // Find the last message to link via [:NEXT]
        OPTIONAL MATCH (i)-[:CONTAINS]->(last:Message)
        WHERE NOT (last)-[:NEXT]->()
```

Step 2: Find the **last** message in the chain. The logic: - `OPTIONAL MATCH` — Try to find it, but don't fail if there are no messages yet (first message in the conversation). - `(i)-[:CONTAINS]->(last:Message)` — Find any Message that this Interaction contains. - `WHERE NOT (last)-[:NEXT]->()` — Filter to only the message that has **no outgoing `[:NEXT]` relationship** — i.e., it's the tail of the linked list.

If this is the first message, `last` will be `NULL`.

```
        CREATE (m:Message {
            message_id: $message_id,
            role: $role,
            content: $content,
            timestamp: $timestamp
        })

        CREATE (i)-[:CONTAINS]->(m)
```

Step 3: Create the new message node and link it to the Interaction. Every message is always linked to its parent Interaction via `[:CONTAINS]`.

```
        // Link to the previous message if it exists (native Cypher conditio
        WITH last, m
        FOREACH (_ IN CASE WHEN last IS NOT NULL THEN [1] ELSE [] END |
            CREATE (last)-[:NEXT]->(m)
        )
```

Step 4: The Conditional Link. This is the most sophisticated part. Let's break it down:

Part	Meaning
`WITH last, m`	Carry forward both variables to the next clause
`CASE WHEN last IS NOT NULL THEN [1] ELSE [] END`	If `last` exists, produce a list with one element `[1]`. If `last` is NULL, produce an empty list `[]`.
`FOREACH (_ IN ... | CREATE ...)`	`FOREACH` iterates over the list. If the list has one element, the `CREATE` executes once. If the list is empty, the `CREATE` never executes.

Why not just `IF`?

Cypher doesn't have a traditional `IF/ELSE` statement. The `FOREACH` + `CASE` pattern is the standard workaround for conditional writes in Cypher. Previously, we used `apoc.do.when()` for this, but that was deprecated in Neo4j 5.x.

Result: The first message in a conversation has no `[:NEXT]` incoming. The second message gets `(msg1)-[:NEXT]->(msg2)`. The third gets `(msg2)-[:NEXT]->(msg3)`. This creates a **singly-linked list** preserving chronological order.

```
        RETURN m.message_id AS id
        """
```

Returns the new message's ID for confirmation.

```
        with get_session() as session:
            result = session.run(
                query,
                session_id=session_id,
                message_id=message_id,
                role=role,
                content=content,
                timestamp=datetime.now().isoformat()
            )
```

```
            record = result.single()
            return record["id"] if record else None
```

`result.single()` — Expects exactly one result record. Returns `None` if the query matched nothing (e.g., invalid session_id).

The Graph After Two Messages

```
(Interaction: session_abc)
    │
    ├──[:CONTAINS]──→ (Message: role="user", content="How does clean archite
    │                     │
    │                     └──[:NEXT]──→ (Message: role="agent", content="By
    │                                       │
    └──[:CONTAINS]──────────────────────────┘
```

Both messages are `[:CONTAINS]` children of the Interaction, AND they're linked sequentially via `[:NEXT]`. This dual-linking gives you both: - **Random access:** Find all messages in a session via `[:CONTAINS]` - **Sequential access:** Walk through them in order via `[:NEXT]`

Module 06: Episodic Memory

File Covered

`src/graph_cortex/core/memory/episodic.py`

What Episodic Memory Does

Episodic Memory compresses raw conversations into **discrete, searchable events**. Instead of storing every message forever, a summary is created: *"User asked about Clean Architecture; agent explained decoupling."* Events are chained chronologically via `[:FOLLOWS]` to create a strict timeline.

Graph structure it creates:

```
(Event_001: "User asked about Clean Architecture...")
    ├── [:SUMMARIZES] → (Interaction: session_abc)
    └── [:FOLLOWS] → (Event_002: "User discussed Neo4j indexes...")
                         └── [:FOLLOWS] → (Event_003: ...)
```

Full Code with Line-by-Line Explanation

```
import uuid
from datetime import datetime
from graph_cortex.infrastructure.db.neo4j_connection import get_session
```

Same imports as Working Memory. The pattern is consistent across all memory layers.

```
class EpisodicMemory:
    """
    Chronological event summaries.
    Compresses working memory into discrete, searchable "events".
    """
    def __init__(self):
        pass
```

Again, stateless service. No instance variables.

The `create_event()` Method

```
    def create_event(self, session_id: str, summary: str):
        """
        Creates an Event node summarizing a specific interaction session and
        chronologically links it to the previous event.
        """
        event_id = str(uuid.uuid4())
```

Generates a unique ID for this event (e.g., `"45a3e13b-a356-41b2-a533-a150efdb9346"`).

```
        query = """
        MATCH (i:Interaction {session_id: $session_id})
```

Step 1: Find the Interaction session that this event will summarise. This is the bridge between Working Memory and Episodic Memory.

```
        // Find the most recent event to link via a chronological chain.
        OPTIONAL MATCH (latest:Event)
```

```
        WHERE NOT (latest)-[:FOLLOWS]->()
```

Step 2: Find the "tail" of the event chain — the most recent event that doesn't yet have a `[:FOLLOWS]` outgoing relationship.

Critical insight: `OPTIONAL MATCH (latest:Event)` searches **all** Event nodes in the entire database (not just ones related to this session). This means the chronological chain spans across ALL sessions. This is intentional — it creates a single global timeline of everything the AI has ever experienced.

If this is the very first event ever created, `latest` will be `NULL`.

```
        CREATE (e:Event {
            event_id: $event_id,
            summary: $summary,
            timestamp: $timestamp
        })
        CREATE (e)-[:SUMMARIZES]->(i)
```

Step 3: Create the Event node and link it to its source Interaction via `[:SUMMARIZES]`. This preserves **provenance** — you can always trace a high-level summary back to the raw conversation that generated it.

```
        // Chronologically chain to previous event (native Cypher conditiona
        WITH latest, e
        FOREACH (_ IN CASE WHEN latest IS NOT NULL THEN [1] ELSE [] END |
            CREATE (latest)-[:FOLLOWS]->(e)
        )
```

Step 4: The same `FOREACH` + `CASE` conditional pattern from Working Memory. If a previous event exists, chain it: `(Event_old)-[:FOLLOWS]->(Event_new)`. If this is the first event, skip.

```
        RETURN e.event_id AS id
        """
```

```
        with get_session() as session:
            result = session.run(
                query,
                session_id=session_id,
                summary=summary,
                event_id=event_id,
                timestamp=datetime.now().isoformat()
            )
            record = result.single()
            return record["id"] if record else None
```

Returns the newly created event's UUID for downstream use (e.g., Semantic Memory needs it to link entities back to this event).

The Chronological Chain

After 3 sessions over 3 days:

```
(Event: April 14, "User discussed Clean Architecture")
    │
    └──[:FOLLOWS]──→ (Event: April 15, "User explored Neo4j indexes")
                          │
                          └──[:FOLLOWS]──→ (Event: April 16, "User upgraded e
```

This gives the AI the ability to: - **Recall recent events:** Walk backwards through `[:FOLLOWS]` chain. - **Understand temporal relationships:** "Clean Architecture was discussed BEFORE Neo4j indexes." - **Answer temporal queries:** "What did we talk about yesterday?"

Relationship Between Working and Episodic Memory

```
Working Memory (raw)                    Episodic Memory (compressed)
────────────────────                    ────────────────────────────
(Interaction: session_abc)              (Event: "User asked about
    ├── Message: "How does..."              Clean Architecture...")
    └── Message: "By strictly..."               │
                                                └── [:SUMMARIZES] → (Interact
```

The `[:SUMMARIZES]` relationship is the bridge. You can always "drill down" from a high-level event summary back into the raw messages. This is how the system supports both: - **Fast, high-level overview:** Query the Event chain - **Deep, detailed inspection:** Follow `[:SUMMARIZES]` → `[:CONTAINS]` → Messages

---

Common Client Questions

Q: "Who generates the summary?"
A: In the current implementation, the summary is provided by the caller (typically an LLM). The `MemoryManager.consolidate_episode()` method receives the summary as a parameter. In production, you'd call GPT/Claude to generate this summary from the raw messages.

Q: "What if the chain breaks — two events are created simultaneously?"
A: The `OPTIONAL MATCH (latest:Event) WHERE NOT (latest)-[:FOLLOWS]->()` pattern finds the tail. If a race condition occurs, both new events would chain to the same predecessor, creating a fork. For a production system, you'd add a transaction lock or use a sequential queue.

Module 07: Semantic Memory & Embeddings

File Covered

`src/graph_cortex/core/memory/semantic.py`

What Semantic Memory Does

Semantic Memory extracts **structured facts** from episodic events. While Episodic Memory captures "what happened," Semantic Memory captures "what we now know." It creates `Entity` nodes (specific things) and `Concept` nodes (abstract ideas), links them with domain-specific relationships, and embeds them as 768-dimensional vectors for semantic search.

Graph structure it creates:

```
(Entity: "Clean Architecture") ──[:IS_A_PATTERN_OF]──> (Concept: "Software D
    │                                                        │
    └── [:EXTRACTED_FROM] → (Event_001)              └── [:EXTRACTED_FROM] →
```

Full Code with Line-by-Line Explanation

```
from typing import List, Dict
from graph_cortex.infrastructure.db.neo4j_connection import get_session
```

```
class SemanticMemory:
    """
    Entity-level abstractions.
    Maps out global knowledge elements derived from specific episodic events
    """
    def __init__(self):
        self.semantic_model = None
```

`self.semantic_model = None` — This is the lazy-loading slot for the SentenceTransformer model. We don't load the model here because: 1. It takes 2-3 seconds and ~500MB of RAM. 2. Some operations might not need embeddings. 3. We only want to load it once and cache it.

The Embedding Engine: `_get_embedding()`

```
    def _get_embedding(self, text: str) -> List[float]:
        """Lazy loads SentenceTransformer using Apple Silicon MPS and return
        if not self.semantic_model:
            from sentence_transformers import SentenceTransformer
            self.semantic_model = SentenceTransformer('BAAI/bge-base-en-v1.5
        return self.semantic_model.encode(text).tolist()
```

Line-by-line:

Line	Explanation
`if not self.semantic_model:`	**Lazy loading guard.** Only loads the model on the very first call. All subsequent calls skip straight to `encode()`.

Line	Explanation
`from sentence_transformers import SentenceTransformer`	**Deferred import.** The `import` is inside the function, not at the top of the file. This means if the `sentence-transformers` package isn't installed, the rest of the file still imports fine — it only fails when you actually try to embed something.
`SentenceTransformer('BAAI/bge-base-en-v1.5', device='mps')`	**Model loading.** Downloads (first time) and loads the BGE-base model with ~109M parameters. `device='mps'` routes all tensor operations through Apple Silicon's Metal Performance Shaders GPU instead of the CPU.
`.encode(text)`	Converts the input string into a 768-dimensional numpy array. Internally, the model tokenises the text, passes it through 12 transformer layers, and mean-pools the output.
`.tolist()`	Converts the numpy array to a plain Python list of floats, which Neo4j can store natively.

What is an embedding?

An embedding is a mathematical representation of meaning. The model converts text like `"Clean Architecture"` into a list of 768 numbers: `[0.0312, -0.0891, 0.1245, ...]`. Texts with similar meanings produce similar number patterns. You can measure this similarity using cosine distance.

Example:

```
"Clean Architecture" → [0.03, -0.09, 0.12, ...]
"Software Design"    → [0.04, -0.08, 0.11, ...]   ← Very similar! (cosine ≈
"Pizza Recipe"       → [-0.21, 0.45, -0.33, ...]  ← Very different! (cosine
```

Method 1: `add_entity()`

```
    def add_entity(self, name: str, node_type: str = "Entity", attributes: D
        """Creates or updates a general semantic entity, embedding it as a v
        if attributes is None:
            attributes = {}
```

Mutable default argument guard. In Python, `def f(x={})` is a famous bug — the empty dict is shared across all calls. Setting it to `None` and creating a new dict inside the function prevents this.

```
        embedding = self._get_embedding(name)
```

Immediate embedding. The moment an entity is created, it gets vectorised. There's no "unembedded" state — every Entity in the database always has an `embedding` property.

```
        query = f"MERGE (e:{node_type} {{name: $name}}) "
```

Dynamic label. The `node_type` parameter defaults to `"Entity"` but can be set to `"Concept"` or any other label. The f-string is used here because **Neo4j parameters cannot be used for labels** — only for property values. This is a Cypher language limitation.

Security note: Since `node_type` comes from internal code (not user input), this is safe. If it were user-facing input, you'd need to whitelist allowed labels.

```
        set_statements = ["e.embedding = $embedding"]
        for k in attributes.keys():
            set_statements.append(f"e.{k} = ${k}")

        query += f"SET {', '.join(set_statements)} "
        query += "RETURN e.name AS name"
```

Dynamic SET construction. Builds a SET clause like:

```
SET e.embedding = $embedding, e.description = $description, e.category = $ca
```

This allows arbitrary additional attributes without modifying the Cypher template.

```
        with get_session() as session:
            session.run(query, name=name, embedding=embedding, **attributes)
        return name
```

`**attributes` unpacks the dictionary as keyword arguments, passing them all to Neo4j as parameters.

Method 2: `extract_from_event()` — The Knowledge Extraction Pipeline

```
    def extract_from_event(self, event_id: str, entity_name: str, concept_na
        """
        Extracts structured semantic knowledge from an event and calculates
        """
        rel_type = relationship_type.upper().replace(" ", "_")
```

Relationship sanitisation. Neo4j relationship types must be uppercase with underscores by convention. `"is a pattern of"` becomes `"IS_A_PATTERN_OF"`.

```
        entity_vector = self._get_embedding(entity_name)
        concept_vector = self._get_embedding(concept_name)
```

Both get embedded. The entity ("Clean Architecture") and the concept ("Software Design") are independently converted into 768-dim vectors.

```
        query = f"""
        MATCH (ev:Event {{event_id: $event_id}})
```

```
        MERGE (e:Entity {{name: $entity_name}})
        SET e.embedding = $entity_vector

        MERGE (c:Concept {{name: $concept_name}})
        SET c.embedding = $concept_vector

        MERGE (e)-[:EXTRACTED_FROM]->(ev)
        MERGE (c)-[:EXTRACTED_FROM]->(ev)
        MERGE (e)-[:{rel_type}]->(c)
        """
```

This single Cypher query does 5 things atomically:

Step	Cypher	What It Does
1	`MATCH (ev:Event {event_id: ...})`	Find the source event
2	`MERGE (e:Entity {name: ...})`	Find or create the Entity node
3	`SET e.embedding = ...`	Store/update the 768-dim vector
4	`MERGE (e)-[:EXTRACTED_FROM]->(ev)`	Link entity to its source event (provenance)
5	`MERGE (e)-[:IS_A_PATTERN_OF]->(c)`	Create the semantic relationship

Why `MERGE` everywhere?

`MERGE` is idempotent. If you call `extract_from_event()` twice with the same entity name, it won't create duplicates — it'll find the existing node and just update the embedding. This makes the system resilient to repeated processing.

Why `f-string` for the relationship type?

Same as labels — Cypher doesn't allow parameterised relationship types. `MERGE (e)-[:$type]->(c)` is invalid Cypher. So we use an f-string with the sanitised `rel_type`.

The Knowledge Graph After Extraction

```
                     (Event_001: "User asked about Clean Architecture...")
                    /              |                  \
          [:EXTRACTED_FROM]  [:EXTRACTED_FROM]    [:EXTRACTED_FROM]
                /                 |                     \
(Entity: "Clean Architecture")  (Concept: "Software Design")  (Entity: "Neo4
         |                                                          |
         └── [:IS_A_PATTERN_OF] → (Concept: "Software Design")     |
                                                                    └── [:BE
```

This creates a rich **knowledge graph** where: - Facts are linked to their sources (provenance tracking). - Entities are connected to abstract concepts. - The AI can traverse from any node to discover related knowledge.

Module 08: Memory Manager Orchestrator

File Covered

`src/graph_cortex/core/memory/manager.py`

What the Memory Manager Does

The `MemoryManager` is the **traffic cop** of the entire memory system. It provides a clean, high-level API that the interface layer calls. Internally, it delegates to the three memory sub-systems in the correct sequence. No external code needs to know about Working, Episodic, or Semantic memory individually.

Full Code with Line-by-Line Explanation

```
from graph_cortex.core.memory.working import WorkingMemory
from graph_cortex.core.memory.episodic import EpisodicMemory
from graph_cortex.core.memory.semantic import SemanticMemory
from graph_cortex.infrastructure.storage.sharding import sharder
from typing import List, Dict
```

Imports all three memory sub-systems plus the property sharder. Notice the imports follow Clean Architecture — core imports core, but also reaches into infrastructure for the sharder (this is the one pragmatic exception where the `manager` uses a thin infrastructure utility).

```
class MemoryManager:
    """
    Orchestrates the Multi-Layered Memory Framework (MLMF).
    Handles the pipeline from raw interaction -> event summarization -> sema
    """
    def __init__(self):
        self.working = WorkingMemory()
        self.episodic = EpisodicMemory()
        self.semantic = SemanticMemory()
```

Composition over Inheritance. The MemoryManager doesn't inherit from any memory class — it *contains* instances of all three. This is the **Facade Pattern** — it hides complexity behind a simple interface.

Method 1: `process_turn()` — Real-Time Ingestion

```
    def process_turn(self, session_id: str, user_input: str, agent_response:
        self.working.add_interaction(session_id)
        self.working.add_message(session_id, role="user", content=user_input
        self.working.add_message(session_id, role="agent", content=agent_res
        return True
```

What happens in sequence:

Step	Method Called	What It Creates
1	`add_interaction(session_id)`	Creates/finds the `Interaction` node for this session
2	`add_message(..., role="user", ...)`	Creates a `Message` node, links it via `[:CONTAINS]` and `[:NEXT]`
3	`add_message(..., role="agent", ...)`	Creates another `Message`, chains it via `[:NEXT]` to the user message

Result in Neo4j:

```
(Interaction) --[:CONTAINS]--> (Message: user) --[:NEXT]--> (Message: agent)
                └──[:CONTAINS]──────────────────────────────────────────┘
```

Important: `process_turn()` only touches **Working Memory**. No summarisation, no knowledge extraction. It's designed to be called in real-time during a conversation.

Method 2: `consolidate_episode()` — Knowledge Extraction Pipeline

```
    def consolidate_episode(self, session_id: str, generated_summary: str, e
```

Parameters: - `session_id` — Which conversation to consolidate - `generated_summary` — A human-readable summary (typically LLM-generated) - `extracted_entities` — A list of structured facts extracted from the conversation

Example `extracted_entities`:

```
[
    {"entity": "Clean Architecture", "concept": "Software Design", "relation
    {"entity": "Neo4j Driver", "concept": "Infrastructure", "relation": "BEL
]
```

```
        # Property Shard the potentially heavy summary
        stored_summary_ref = sharder.store(f"ep_{session_id}", generated_sum
```

Step 1: Property Sharding. The summary text can be very long. Rather than storing heavy text directly on the graph node (which slows down traversal), we pass it through the sharder. Currently in "local" mode, the sharder just returns the string unchanged. In production with `mode="s3"`, it would upload the text to S3 and return a lightweight URI like `s3://ns-dmg-shard/ep_session_abc`.

```
        # Create Episodic Event
        event_id = self.episodic.create_event(session_id, stored_summary_ref
```

Step 2: Episodic Memory. Creates the Event node with the summary, links it to the Interaction via `[:SUMMARIZES]`, and chains it to the previous event via `[:FOLLOWS]`.

```
        # Extract into Semantic Memory layer
        for item in extracted_entities:
            self.semantic.add_entity(item["entity"])
            self.semantic.add_entity(item["concept"])
            self.semantic.extract_from_event(
                event_id=event_id,
                entity_name=item["entity"],
                concept_name=item["concept"],
                relationship_type=item.get("relation", "RELATED_TO")
            )
```

Step 3: Semantic Memory. For each extracted fact: 1. `add_entity(item["entity"])` — Ensures the Entity node exists and has a 768-dim embedding. 2. `add_entity(item["concept"])` — Same for the Concept node. 3. `extract_from_event(...)` — Links both to the source Event and creates the domain relationship between them.

Why `add_entity()` before `extract_from_event()`?
`extract_from_event()` uses `MERGE`, which also creates nodes. But calling `add_entity()` first ensures the embedding is set even if `extract_from_event()` is called with different parameters later. It's a belt-and-suspenders approach.

```
        return event_id
```

Returns the Event ID so the caller can reference it.

The Complete Pipeline Visualised

```
process_turn()                          consolidate_episode()
══════════════                          ═══════════════════════

User says: "How does                    LLM generates:
clean architecture..."                  Summary: "User asked about
                │                       Clean Architecture..."
                ▼                               │
        ┌─────────────────┐                     ▼
        │  WORKING        │             ┌─────────────────┐
        │  MEMORY         │             │  PROPERTY       │
        │                 │             │  SHARDER        │
        │ Interaction     │             │ (offload text)│
        │    └─ Msg 1     │             └─────────────────┘
        │       └─ Msg 2│                     │
        └─────────────────┘                     ▼
                                        ┌─────────────────┐
                                        │  EPISODIC       │
                                        │  MEMORY         │
                                        │                 │
                                        │ Event ─FOLLOWS─> prev
                                        │   └─SUMMARIZES─> Interaction
                                        └─────────────────┘
                                                │
                                                ▼
                                        ┌─────────────────┐
                                        │  SEMANTIC       │
                                        │  MEMORY         │
                                        │                 │
                                        │ Entity ──REL──> Concept
                                        │   └─EXTRACTED_FROM─> Event
                                        └─────────────────┘
```

Common Client Questions

Q: "Why are these two separate methods instead of one?"

A: In a real-time chat application, `process_turn()` is called on every single message exchange (milliseconds matter). `consolidate_episode()` is called at the end of a conversation or periodically — it's more expensive because it runs the embedding model, generates summaries, and creates semantic links. Separating them lets you handle real-time ingestion at full speed without blocking on ML inference.

Q: "Where does the summary come from?"

A: The `generated_summary` parameter is expected to come from an LLM. In a production pipeline, you'd call something like `GPT-4o: Summarize this conversation in one sentence` and pass the result here.

Module 09: Property Sharding

File Covered

`src/graph_cortex/infrastructure/storage/sharding.py`

What Property Sharding Solves

In a graph database, performance depends on how fast you can traverse relationships. When you run a BFS (Breadth-First Search) across 1000 nodes, Neo4j loads each node's properties into memory. If every Event node carries a 10KB summary string, traversal loads 10MB of text you might not even need.

Property Sharding offloads heavy properties (like text summaries) to cheaper storage, keeping the graph lightweight.

Full Code with Line-by-Line Explanation

```
class PropertySharder:
    """
    Experimental interface for property sharding in Phase 1.
    Offloads heavy properties away from graph topology to maintain performan
    """
    def __init__(self, mode="local"):
        self.mode = mode
```

Two modes: - `"local"` — Passthrough mode. The text stays inline on the Neo4j node. Used for development. - `"s3"` (or any external mode) — Would upload text to S3/MongoDB and store only a lightweight URI reference on the node.

```
    def store(self, node_id: str, payload: str) -> str:
        """
        Stores heavy payload and returns a lightweight reference.
        """
        if self.mode == "local":
            return payload
        else:
            ref_uri = f"s3://ns-dmg-shard/{node_id}"
            return ref_uri
```

In local mode: Simply returns the original string. The Event node stores the full summary text directly. Zero overhead for development.

In S3 mode: Would upload the payload to an S3 bucket and return only the URI string `s3://ns-dmg-shard/ep_session_abc`. The Event node would store this tiny reference instead of the full text. When you need the actual text, you call `retrieve()`.

```
    def retrieve(self, ref_uri: str) -> str:
        """
        Retrieves the heavy payload from the shard.
        """
        if self.mode == "local":
            return ref_uri
        return f"Loaded external content for {ref_uri}"
```

The inverse operation. In local mode, the "reference" IS the content, so just return it.

```
sharder = PropertySharder()
```

Module-level singleton instance. All code imports and uses this single `sharder` object.

Why This Pattern Matters for Clients

Q: "What happens when the graph grows to millions of nodes?"
A: *"We've already abstracted property sharding. Heavy text properties can be transparently offloaded to S3 or MongoDB by switching one configuration flag, keeping graph traversal at O(1) per node regardless of text length."*

Q: "Is S3 mode actually implemented?"
A: *"The abstraction is in place. The S3 upload/download calls would be added to* `store()` *and* `retrieve()`*. The important thing is that no other code in the system needs to change — the MemoryManager doesn't know or care where the text lives."*

This is the **Dependency Inversion Principle** in action.

Module 10: Retrieval Queries (Raw Cypher)

File Covered

`src/graph_cortex/infrastructure/db/queries/retrieval_queries.py`

Why This File Exists Separately

In Clean Architecture, the **infrastructure layer** contains all database-specific code. The `RetrievalEngine` (core) doesn't write Cypher — it calls functions in this file and receives plain Python dictionaries back. If you migrate from Neo4j to AWS Neptune (which uses Gremlin instead of Cypher), you only rewrite this file.

Query 1: `get_anchor_nodes_by_name()` — The Lexical Trigger

```
def get_anchor_nodes_by_name(session, entity_names, limit=5):
    """
    Finds anchor nodes (Entities or Concepts) matching specific string names
    This acts as the Lexical (BM25-style) trigger in the Dual-Trigger initia
    """
```

```
    query = """
    UNWIND $names AS name
    MATCH (n) WHERE (n:Entity OR n:Concept) AND toLower(n.name) CONTAINS toL
```

```
    RETURN elementId(n) AS node_id, n.name AS name, labels(n)[0] AS type
    LIMIT $limit
    """
```

Cypher breakdown:

Line	What It Does
`UNWIND $names AS name`	Takes the input list `["System Design", "Clean Architecture"]` and iterates over it, processing each term one at a time.
`MATCH (n)`	Find any node `n` in the database.
`WHERE (n:Entity OR n:Concept)`	Filter to only Entity and Concept nodes. Ignore Message, Event, Interaction nodes.
`AND toLower(n.name) CONTAINS toLower(name)`	**Case-insensitive substring matching.** `"clean architecture"` matches `"Clean Architecture"`. This is the "lexical" trigger — pure string matching, no ML.
`elementId(n) AS node_id`	Returns Neo4j's internal element ID (used for subsequent traversal).
`labels(n)[0] AS type`	Returns the node's first label (`"Entity"` or `"Concept"`).
`LIMIT $limit`	Cap results to prevent returning thousands of matches.

```
    result = session.run(query, names=entity_names, limit=limit)
    return [record.data() for record in result]
```

Converts Neo4j records to Python dicts: `[{"node_id": "4:abc:7", "name": "Clean Architecture", "type": "Entity"}]`

When this query FAILS to find matches (lexical miss), the Retrieval Engine falls back to semantic vector search.

Query 2: `execute_spreading_activation_hop()` — The BFS Traversal

```
def execute_spreading_activation_hop(session, target_node_id, hop_depth):
    """
    Executes a custom Cypher BFS traversal from the target node up to a cert
    Calculates raw 'degree' for downstream fan-effect attenuation.
    Note: Neo4j 5.x does not allow parameters in variable-length patterns,
    so hop_depth is safely interpolated as an integer literal.
    """
    depth = int(hop_depth)  # Sanitize to prevent injection
```

Security: `int()` ensures only an integer value is used. If someone somehow passes `"3; DROP DATABASE"`, `int()` throws a `ValueError` before it reaches Neo4j.

```
    query = f"""
    MATCH path = (start)-[*1..{depth}]-(connected)
    WHERE elementId(start) = $node_id
```

Line-by-line:

Part	Meaning
`MATCH path = (start)-[*1..{depth}]-(connected)`	Find all paths from `start` to any `connected` node, following between 1 and `{depth}` relationships (default: 3). The `-` (no arrow) means **any direction** — both incoming and outgoing relationships.
`path = ...`	Assigns the entire path to a variable so we can calculate distance from it.
`[*1..3]`	Variable-length relationship pattern. `*1..3` means "at least 1 hop, at most 3 hops."
`elementId(start) = $node_id`	Start from the specific anchor node identified by its element ID.

```
    WITH start, connected, REDUCE(s = 0, n IN nodes(path) | s + 1) AS distan
         COUNT { (connected)--() } AS degree
```

Part	Meaning
`REDUCE(s = 0, n IN nodes(path) | s + 1)`	Counts the number of nodes in the path. This is the **distance** from the anchor. A node 2 hops away has `distance = 3` (start + 2 intermediate + target).
`COUNT { (connected)--() }`	**Neo4j 5.x syntax.** Counts how many relationships the connected node has (its **degree**). High-degree nodes are "hubs" — they connect to many things and should be penalised.

Why `COUNT { }` instead of `SIZE()`?

Neo4j 5.x deprecated `SIZE((pattern))` for non-boolean contexts. The new `COUNT { pattern }` subquery syntax is the modern replacement.

```
    RETURN
        elementId(connected) AS node_id,
        connected.name AS name,
        labels(connected)[0] AS type,
        distance,
        degree
    ORDER BY distance ASC
    """
    result = session.run(query, node_id=target_node_id)
    return [record.data() for record in result]
```

Returns a list of dictionaries, sorted by distance (closest nodes first):

```
[
    {"node_id": "4:abc:8", "name": "Software Design", "type": "Concept", "di
    {"node_id": "4:abc:9", "name": "Neo4j Driver", "type": "Entity", "distan
    ...
]
```

The `distance` and `degree` values are passed to the Lateral Inhibition formula to calculate activation energy.

Query 3: `get_anchors_by_vector_similarity()` — The Semantic Vector Fallback

```
def get_anchors_by_vector_similarity(session, vector, limit=2):
    """
    Finds anchor nodes based on semantic vector similarity (Cosine).
    Queries the 'entity_vector_index' initialized in schema_migrations.
    """
    query = """
    CALL db.index.vector.queryNodes('entity_vector_index', $limit, $vector)
    YIELD node, score
    WHERE score > 0.65
    RETURN elementId(node) AS node_id, node.name AS name, labels(node)[0] AS
    ORDER BY score DESC
    """
```

Line-by-line:

Part	Meaning
`CALL db.index.vector.queryNodes(...)`	Neo4j's native vector search procedure. Takes the index name, the number of results, and the query vector.
`'entity_vector_index'`	References the index created in `schema_migrations.py`.
`$limit`	How many nearest neighbors to return (default: 2).
`$vector`	The 768-dimensional query vector (computed from the user's search term).

Part	Meaning
`YIELD node, score`	The procedure returns the matched node and its cosine similarity score (0-1).
`WHERE score > 0.65`	**Similarity threshold.** Filters out weak matches. 0.65 means the vectors must share at least 65% cosine similarity. This prevents the engine from "hallucinating" irrelevant anchors.
`ORDER BY score DESC`	Best matches first.

```
    result = session.run(query, limit=limit, vector=vector)
    return [record.data() for record in result]
```

Returns:

```
[
    {"node_id": "4:abc:7", "name": "Software Design", "type": "Entity", "sco
    {"node_id": "4:abc:6", "name": "Clean Architecture", "type": "Entity", "
]
```

How All Three Queries Connect

```
User query: ["System Design"]
                │
                ▼
    ┌── get_anchor_nodes_by_name() ──┐
    │   "System Design" CONTAINS?    │
    │   Result: [] (empty - no match)│
    └────────────┬────────────────────┘
                 │ MISS
                 ▼
```

```
┌── get_anchors_by_vector_similarity() ──┐
│   Encode "System Design" → 768-dim     │
│   Cosine search the vector index        │
│   Result: ["Software Design" (0.95),    │
│            "Clean Architecture" (0.82)] │
└──────────────┬──────────────────────────┘
               │ HIT
               ▼
┌── execute_spreading_activation_hop() ──┐
│   BFS from "Software Design" (3 hops)  │
│   BFS from "Clean Architecture" (3 hops)│
│   Returns distance + degree for each    │
└─────────────────────────────────────────┘
```

Module 11: Lateral Inhibition — The Math

File Covered

`src/graph_cortex/core/retrieval/inhibition.py`

The Neuroscience Behind It

Lateral Inhibition is a real phenomenon in the human brain. When one neuron fires strongly, it suppresses the firing of its neighbours. This prevents the brain from being overwhelmed by noise and forces it to focus on the most relevant signals.

The Hub Explosion Problem:
In a knowledge graph, some nodes are "hubs" — they connect to everything. For example, a node labelled `"User"` might be linked to every single interaction. If Spreading Activation reaches `"User"`, it would fan out to thousands of nodes, flooding the AI's context window with irrelevant garbage.

Lateral Inhibition mathematically suppresses these hubs.

The Energy Decay Formula

```
AE = initial_energy / (((distance × distance_penalty) + 1) × ((degree × degr
```

What Each Variable Means

Variable	Default	What It Controls
`initial_energy`	`1.0`	The starting energy of an anchor node (maximum = 1.0).
`distance`	varies	How many hops this node is from the anchor (returned by BFS query).
`distance_penalty`	`0.5`	How aggressively energy decays with distance. Higher = faster decay.
`degree`	varies	How many relationships this node has (returned by BFS query).
`degree_penalty`	`0.1`	How aggressively energy decays for hub nodes. Higher = harsher on hubs.
`cutoff_threshold`	`0.2`	Minimum energy required to survive. Below this = inhibited/dropped.

Why `+1` in the Formula?

Without `+1`, the formula would be:

```
AE = 1.0 / ((distance × 0.5) × (degree × 0.1))
```

Problem: If `distance = 0` (the anchor itself), the denominator becomes 0, causing a **division by zero**. Adding `+1` ensures the denominator is always at least 1.

For the anchor node itself (distance=0, degree=1):

```
AE = 1.0 / (((0 × 0.5) + 1) × ((1 × 0.1) + 1))
AE = 1.0 / (1 × 1.1)
AE = 0.909   (Near-maximum energy ✓)
```

Full Code with Line-by-Line Explanation

```
def apply_lateral_inhibition(traversed_nodes, initial_energy=1.0, degree_pen
    """
    Simulates lateral inhibition and the Fan Effect computationally.
    Calculates the Activation Energy (AE) for each node.
    AE = initial_energy / (((distance * distance_penalty) + 1) * ((degree *
    Nodes whose AE falls below the cutoff_threshold are inhibited/dropped.
    """
```

Pure function. No database imports, no side effects, no state. Given the same input, it always produces the same output. This is a core principle of Clean Architecture — the math is testable without any infrastructure.

```
    filtered_nodes = []
    dropped_hubs = []
```

Two output lists: nodes that survive the filter, and nodes that get inhibited.

```
    for node_data in traversed_nodes:
        degree = node_data.get("degree", 1)
        distance = node_data.get("distance", 1)
```

`.get("degree", 1)` — Safe dictionary access with a default value. If a node somehow doesn't have a `degree` field, it defaults to 1 (no penalty).

```
        # Calculate Activation Energy (Decay Formula)
        denominator = ((distance * distance_penalty) + 1) * ((degree * degre
        activation_energy = initial_energy / denominator
```

The core math. Let's trace through multiple examples:

```
        node_data["activation_energy"] = round(activation_energy, 4)
```

Mutates the input dictionary by adding an `activation_energy` field. Rounded to 4 decimal places for readability. This value gets passed back to the Retrieval Engine and included in the final output.

```
        if activation_energy >= cutoff_threshold:
            filtered_nodes.append(node_data)
        else:
            dropped_hubs.append(node_data.get("name", "UnknownHub"))
```

The binary decision: survive or die. If `AE >= 0.2`, the node is relevant enough to keep. Otherwise, it's logged as an inhibited hub and dropped.

```
    return filtered_nodes, dropped_hubs
```

Returns both lists so the caller can report what was kept and what was dropped.

Worked Examples with Real Data

Example 1: Close, Specific Node (SURVIVES ✅)

```
Node: "Software Design" (distance=2, degree=4)

denominator = ((2 × 0.5) + 1) × ((4 × 0.1) + 1)
            = (1 + 1) × (0.4 + 1)
            = 2 × 1.4
            = 2.8

AE = 1.0 / 2.8 = 0.3571

0.3571 >= 0.2? YES → SURVIVES ✅
```

Example 2: Far, Specific Node (SURVIVES ✔)

```
Node: "Neo4j Driver" (distance=3, degree=2)

denominator = ((3 × 0.5) + 1) × ((2 × 0.1) + 1)
            = (1.5 + 1) × (0.2 + 1)
            = 2.5 × 1.2
            = 3.0

AE = 1.0 / 3.0 = 0.3333

0.3333 >= 0.2? YES → SURVIVES ✔
```

Example 3: Close Hub Node (BORDERLINE ⚠)

```
Node: "Project" (distance=1, degree=15)

denominator = ((1 × 0.5) + 1) × ((15 × 0.1) + 1)
            = (0.5 + 1) × (1.5 + 1)
            = 1.5 × 2.5
            = 3.75

AE = 1.0 / 3.75 = 0.2667

0.2667 >= 0.2? YES → BARELY SURVIVES ⚠
```

Example 4: Far Hub Node (INHIBITED ✖)

```
Node: "User" (distance=3, degree=50)

denominator = ((3 × 0.5) + 1) × ((50 × 0.1) + 1)
            = (1.5 + 1) × (5 + 1)
            = 2.5 × 6.0
            = 15.0

AE = 1.0 / 15.0 = 0.0667
```

```
0.0667 >= 0.2? NO → INHIBITED ✗
```

Example 5: Distant Generic Concept (INHIBITED ✗)

```
Node: "Technology" (distance=4, degree=30)

denominator = ((4 × 0.5) + 1) × ((30 × 0.1) + 1)
            = (2 + 1) × (3 + 1)
            = 3.0 × 4.0
            = 12.0

AE = 1.0 / 12.0 = 0.0833

0.0833 >= 0.2? NO → INHIBITED ✗
```

Visualising the Filter

```
                    ANCHOR: "Clean Architecture" (AE = 1.0) ✓
                   /                    \
         distance=2                   distance=2
              /                            \
"Software Design" (AE=0.36) ✓    "Event_001" (AE=0.33) ✓
              |                            |
         distance=3                   distance=3
              |                            |
"Neo4j Driver" (AE=0.33) ✓      "User" (degree=50, AE=0.07) ✗ INHIBITED
                                           |
                                      distance=4
                                           |
                                 "Technology" (AE=0.08) ✗ INHIBITED
```

The hub nodes (`"User"` , `"Technology"`) are automatically suppressed, keeping the AI's context window focused on relevant, specific knowledge.

Tuning Guide

If you want...	Adjust...
More aggressive hub suppression	Increase `degree_penalty` (e.g., 0.2)
Deeper traversal reach	Decrease `distance_penalty` (e.g., 0.3)
Keep more nodes	Decrease `cutoff_threshold` (e.g., 0.1)
Keep fewer nodes	Increase `cutoff_threshold` (e.g., 0.3)
More energy at the anchor	Increase `initial_energy` (e.g., 2.0)

Module 12: Retrieval Engine — The Brain

File Covered

`src/graph_cortex/core/retrieval/engine.py`

What the Retrieval Engine Does

The `RetrievalEngine` is the **cognitive core** of GraphCortex. When the AI needs to recall information, this engine:

1. **Finds anchor nodes** using dual triggers (lexical string match → semantic vector fallback)
2. **Fans outward** from anchors using BFS traversal (Spreading Activation)
3. **Calculates energy decay** for every discovered node (Lateral Inhibition)
4. **Deduplicates** overlapping networks when multiple anchors activate
5. **Returns** a clean, filtered knowledge sub-graph

Full Code with Line-by-Line Explanation

Imports

```
from graph_cortex.infrastructure.db.neo4j_connection import get_session
from graph_cortex.infrastructure.db.queries.retrieval_queries import get_anc
from graph_cortex.core.retrieval.inhibition import apply_lateral_inhibition
```

```
from graph_cortex.config.logger import get_retrieval_logger
from sentence_transformers import SentenceTransformer
```

Import	Layer	Purpose
`get_session`	Infrastructure	Neo4j session factory
`get_anchor_nodes_by_name`	Infrastructure	Lexical Cypher query
`execute_spreading_activation_hop`	Infrastructure	BFS Cypher query
`get_anchors_by_vector_similarity`	Infrastructure	Vector search Cypher query
`apply_lateral_inhibition`	Core	Energy decay math
`get_retrieval_logger`	Config	Timestamped log writer
`SentenceTransformer`	External	Embedding model for vector fallback

Class Initialization

```
class RetrievalEngine:
    """
    Orchestrates the Spreading Activation Retrieval process.
    Leverages Lexical/Semantic triggers to find anchors, and transverses out
    """
    def __init__(self, cutoff_threshold=0.2, max_depth=3):
        self.cutoff_threshold = cutoff_threshold
        self.max_depth = max_depth
        self.semantic_model = None  # Lazy load SentenceTransformer here whe
        self.logger = get_retrieval_logger()
```

Attribute	Default	Purpose
`cutoff_threshold`	`0.2`	Minimum activation energy to keep a node (passed to Lateral Inhibition)

Attribute	Default	Purpose
`max_depth`	`3`	Maximum BFS hop depth for Spreading Activation
`semantic_model`	`None`	Lazy-loaded embedding model (only loaded when lexical search fails)
`logger`	Logger instance	Writes timestamped events to `/Logs/retrieval_TIMESTAMP.log`

The `retrieve()` Method — Step by Step

Step 1A: Lexical Trigger

```
    def retrieve(self, query_terms: list):
        # Step 1A: Lexical Trigger
        with get_session() as session:
            anchors = get_anchor_nodes_by_name(session, query_terms)
```

First attempt: Try to find nodes whose names contain the query string. If the user searches for `["Clean Architecture"]`, this will find the Entity directly.

Result: A list of anchor dicts, or an empty list if nothing matched.

Step 1B: Semantic Vector Fallback

```
            # Step 1B: Semantic Vector Fallback
            if not anchors:
                self.logger.info(f"Lexical Miss for '{query_terms}'. Initiat
                print(f"\n[!] Lexical miss for '{query_terms}'. Activating S
```

The fallback trigger. If lexical search found nothing (user searched `["System Design"]` but no node has that exact name), we switch to semantic search.

The event is both: - **Logged** to the `/Logs` file for post-mortem analysis - **Printed** to stdout for immediate feedback

```
                if not self.semantic_model:
                    self.semantic_model = SentenceTransformer('BAAI/bge-base
```

Lazy loading. The model is only loaded when actually needed (lexical miss). If lexical search always succeeds, the model never loads — saving 2-3 seconds of startup time and ~500MB of RAM.

`device='mps'` routes computation through Apple Silicon's Metal Performance Shaders GPU.

```
                vector = self.semantic_model.encode(query_terms[0]).tolist()
                anchors = get_anchors_by_vector_similarity(session, vector,
```

1. Encode the first query term into a 768-dimensional vector.
2. Pass it to Neo4j's native vector index for cosine similarity search.
3. Returns up to 2 anchors that exceed the 0.65 similarity threshold.

```
                if not anchors:
                    self.logger.warning(f"Semantic Fallback Miss for '{query
                    return {"status": "Miss", "anchors": [], "network": [],
                else:
                    self.logger.info(f"Semantic Fallback Success! Found sema
            else:
                self.logger.info(f"Lexical Hit for '{query_terms}'. Found ex
```

Three possible outcomes: 1. **Lexical Hit** → Anchors found by string matching → Continue 2. **Semantic Hit** → Anchors found by vector similarity → Continue
3. **Total Miss** → Neither method found anything → Return empty result immediately

Step 2: Spreading Activation from Anchors

```
            # Step 2: Spreading Activation from Anchors
            activated_network = []
            dropped = []

            for anchor in anchors:
                # Add the anchor itself to the network with maximum energy
                activated_network.append({
                    "node_id": anchor["node_id"],
                    "name": anchor["name"],
                    "type": anchor["type"],
                    "distance": 0,
                    "degree": 1,
                    "activation_energy": 1.0  # Max energy for exact match a
                })
```

The anchor gets maximum energy (1.0). It's distance 0 (it IS the starting point) and degree 1 (we don't penalise the anchor itself for being connected).

```
                # Traverse outwards (Fan out) up to depth limit
                traversed = execute_spreading_activation_hop(session, anchor
```

BFS explosion. From the anchor, find all nodes within 3 hops. This will return dozens to hundreds of nodes, depending on graph density.

Step 3: Lateral Inhibition (Energy Decay)

```
                # Step 3: Apply Lateral Inhibition (Energy Decay)
                filtered, hubs = apply_lateral_inhibition(
                    traversed,
                    cutoff_threshold=self.cutoff_threshold
                )

                activated_network.extend(filtered)
                dropped.extend(hubs)
```

Pass the raw traversed nodes through the energy decay formula. Only nodes with `AE >= 0.2` survive. Hubs are logged as dropped.

Step 4: Network Deduplication

```
# Deduplicate by node_id, keeping the highest activation energy
unique_network_dict = {}
for node in activated_network:
    n_id = node["node_id"]
    if n_id not in unique_network_dict or node["activation_energ
        unique_network_dict[n_id] = node
```

Why deduplication is necessary:

When multiple anchors activate (e.g., both "Software Design" and "Clean Architecture"), their BFS traversals overlap. The same node might appear in both networks. We keep the instance with the **highest activation energy** — the one with the strongest relevance signal.

Example:

```
From "Software Design" BFS: {"name": "Event_001", "AE": 0.33}
From "Clean Architecture" BFS: {"name": "Event_001", "AE": 0.36}

→ Keep the one with AE = 0.36
```

Return the Final Result

```
return {
    "status": "Hit",
    "anchors": [a["name"] for a in anchors],
    "network": list(unique_network_dict.values()),
    "inhibited_hubs": list(set(dropped))
}
```

Field	Type	Content
`status`	`"Hit"` or `"Miss"`	Whether any anchors were found

Field	Type	Content
`anchors`	`["Software Design", "Clean Architecture"]`	Names of the anchor nodes that initiated activation
`network`	List of node dicts	The full activated, filtered, deduplicated knowledge sub-graph
`inhibited_hubs`	`["User", "Technology"]`	Names of nodes that were suppressed by Lateral Inhibition

---

The Complete Retrieval Pipeline

```
User: "System Design"
         │
         ▼
┌─ STEP 1A: LEXICAL TRIGGER ──────────────────────┐
│ Query: MATCH (n) WHERE n.name CONTAINS          │
│        "System Design"                          │
│ Result: [] (MISS — no exact match)              │
└─────────────────────────┬───────────────────────┘
                          │
                          ▼
┌─ STEP 1B: SEMANTIC VECTOR FALLBACK ─────────────┐
│ Encode "System Design" → 768-dim vector (MPS)   │
│ Query Neo4j vector index (cosine search)        │
│ Result: ["Software Design" (0.95),              │
│          "Clean Architecture" (0.82)]           │
└─────────────────────────┬───────────────────────┘
                          │
         ┌────────────────┴────────────────┐
         │                                 │
         ▼                                 ▼
┌─ STEP 2: BFS FROM ──┐   ┌─ STEP 2: BFS FROM ──────┐
```

```
│ "Software Design"   │  │ "Clean Architecture"     │
│ Depth: 3 hops       │  │ Depth: 3 hops            │
│ Discovers: 15 nodes │  │ Discovers: 18 nodes      │
└─────────┬───────────┘  └──────────┬───────────────┘
          │                         │
          ▼                         ▼
┌─ STEP 3: LATERAL INHIBITION ──────────────────────┐
│ For each discovered node:                         │
│   AE = 1.0 / (dist_factor × degree_factor)        │
│   Keep if AE >= 0.2, drop otherwise               │
│                                                   │
│ Kept: 12 nodes    Dropped: 21 hubs                │
└───────────────────────┬───────────────────────────┘
                        │
                        ▼
┌─ STEP 4: DEDUPLICATE ─────────────────────────────┐
│ Merge overlapping networks                        │
│ Keep highest AE for each unique node_id           │
│                                                   │
│ Final network: 10 unique nodes                    │
└───────────────────────┬───────────────────────────┘
                        │
                        ▼
              RETURN TO CALLER
```

Module 13: Logger Configuration

File Covered

`src/graph_cortex/config/logger.py`

What the Logger Does

Creates a dedicated, timestamped log file every time the `RetrievalEngine` runs. This provides a permanent audit trail of: - When the Lexical trigger hit vs. missed - When the Semantic Vector Fallback was activated - What anchors were found and their similarity scores

Full Code with Line-by-Line Explanation

```
import os
import logging
from datetime import datetime
```

- `os` — File path construction and directory creation.
- `logging` — Python's built-in structured logging framework.
- `datetime` — Timestamp generation for unique log filenames.

```
# Build absolute path to the Logs directory at the project root
PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "../.
```

```
LOGS_DIR = os.path.join(PROJECT_ROOT, "Logs")
```

Path resolution explained:

```
__file__              = .../src/graph_cortex/config/logger.py
os.path.dirname(...)  = .../src/graph_cortex/config/
"../../../"           = .../src/graph_cortex/config/ → config/ → graph_corte
os.path.abspath(...)  = /Users/.../Developer/GraphCortex/
```

`LOGS_DIR` becomes `/Users/.../Developer/GraphCortex/Logs/`.

```
# Ensure the Logs directory physically exists early
if not os.path.exists(LOGS_DIR):
    os.makedirs(LOGS_DIR)
```

Module-level execution. This runs when the file is first imported. If the `Logs/` directory doesn't exist, it's created automatically. `os.makedirs()` creates any missing intermediate directories too.

The `get_retrieval_logger()` Function

```
def get_retrieval_logger():
    """
    Configures and returns a dedicated logger for Retrieval operations.
    Maintains a physical trail of Lexical vs Semantic Fallback triggers.
    """
    logger = logging.getLogger("RetrievalEngine")
```

Named logger. `logging.getLogger("RetrievalEngine")` returns a logger specific to the retrieval engine. Python's logging module uses a singleton pattern — calling `getLogger("RetrievalEngine")` anywhere in the codebase always returns the same logger instance.

```
    # Only configure if it doesn't already have handlers to prevent duplicat
    if not logger.handlers:
        logger.setLevel(logging.INFO)
```

Idempotency guard. If `get_retrieval_logger()` is called multiple times (e.g., creating multiple `RetrievalEngine` instances), we don't want to add duplicate file handlers. The `if not logger.handlers` check ensures configuration happens only once.

`logging.INFO` means it captures `INFO`, `WARNING`, `ERROR`, and `CRITICAL` messages. It ignores `DEBUG`.

```
        # Create a timestamped log file specific to this run
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        log_file = os.path.join(LOGS_DIR, f"retrieval_{timestamp}.log")
```

Each run of the application gets its own log file. Example: `retrieval_20260416_184741.log`. This means you can compare logs from different runs side by side.

```
        file_handler = logging.FileHandler(log_file)
        file_handler.setLevel(logging.INFO)
```

FileHandler writes log messages to the specified file. Each handler can have its own level — here it matches the logger's level.

```
        # Structural logging format for easy analysis
        formatter = logging.Formatter('[%(asctime)s] [%(levelname)s] - %(mes
        file_handler.setFormatter(formatter)
```

Log format:

```
[2026-04-16 18:47:41] [INFO] - Lexical Miss for '['System Design']'. Initiat
[2026-04-16 18:47:43] [INFO] - Semantic Fallback Success! Found semantic anc
```

Token	Example	Meaning
`%(asctime)s`	`2026-04-16 18:47:41`	Timestamp when the event occurred
`%(levelname)s`	`INFO`	Severity level
`%(message)s`	`Lexical Miss for...`	The actual log message

```
        logger.addHandler(file_handler)

    return logger
```

Attaches the handler to the logger and returns it. All subsequent `logger.info(...)` calls will write to this file.

Example Log Output

```
[2026-04-16 18:47:41] [INFO] - Lexical Miss for '['System Design']'. Initiat
[2026-04-16 18:47:43] [INFO] - Semantic Fallback Success! Found semantic anc
```

This log tells you exactly: 1. **18:47:41** — Lexical search failed (no exact string match for "System Design") 2. **18:47:43** — 2 seconds later, the semantic fallback found 2 anchors with high cosine scores

The 2-second gap is the time taken to lazy-load the `bge-base-en-v1.5` model on MPS and compute the embedding.

Module 14: CLI Entry Point

File Covered

`src/graph_cortex/interfaces/cli/main.py`

What This File Does

This is the **interface layer** — where the outside world enters GraphCortex. It wires together the infrastructure (schema), core (memory + retrieval), and config (logger) into an end-to-end executable demonstration.

This file serves two purposes: 1. **Verification script** — Proves the entire system works end-to-end. 2. **Reference implementation** — Shows exactly how to use the GraphCortex API.

Full Code with Line-by-Line Explanation

Imports

```
import uuid
import sys
import json
from graph_cortex.infrastructure.db.schema_migrations import initialize_sche
```

```
from graph_cortex.core.memory.manager import MemoryManager
from graph_cortex.core.retrieval.engine import RetrievalEngine
```

Import	Layer	Purpose
`initialize_schema`	Infrastructure	Creates database constraints + vector indexes
`MemoryManager`	Core	Memory orchestrator API
`RetrievalEngine`	Core	Retrieval orchestrator API

Notice how the CLI **only** imports from `infrastructure` (for schema setup) and `core` (for everything else). It never touches the database directly.

The `main()` Function

Phase 0: Database Setup

```
def main():
    print("Welcome to GraphCortex CLI Native Interface.")
    print("Initializing Database Schema...")

    # This might fail gracefully if Neo4j isn't active
    initialize_schema()
```

Schema initialization. Creates all constraints, indexes, and vector indexes. Safe to call multiple times (all queries use `IF NOT EXISTS`). If Neo4j is offline, it prints a warning but doesn't crash.

Phase 1: Working Memory — Ingesting a Conversation

```
    print("\nStarting memory integration...")
    manager = MemoryManager()

    session_id = f"session_{uuid.uuid4().hex[:8]}"
```

Creates a unique session ID like `session_82dd0f7b`. The `hex[:8]` gives us the first 8 hex characters — short enough to read, unique enough to avoid collisions.

```
    print(f"\n[Working Memory] Processing conversation turn for session: {se
    manager.process_turn(
        session_id=session_id,
        user_input="How does clean architecture improve graph database acces
        agent_response="By strictly decoupling the Neo4j driver queries (inf
    )
```

This single call creates 3 nodes and 4 relationships:

```
(Interaction: session_82dd0f7b)
    ├──[:CONTAINS]──→ (Message: role=user, "How does clean architecture...")
    │                     └──[:NEXT]──→
    └──[:CONTAINS]──→ (Message: role=agent, "By strictly decoupling...")
```

Phase 1→2 Bridge: Episodic + Semantic Memory

```
    print("\n[Episodic & Semantic Memory] Consolidating episode...")
    summary = "User asked about Clean Architecture in graphs; agent explaine
    extracted_entities = [
        {"entity": "Clean Architecture", "concept": "Software Design", "rela
        {"entity": "Neo4j Driver", "concept": "Infrastructure", "relation":
    ]

    event_id = manager.consolidate_episode(session_id, summary, extracted_en
    print(f"Episode consolidated with Event ID: {event_id}")
```

This single call triggers a cascade: 1. Property Sharder stores the summary text 2. Episodic Memory creates an Event node and chains it chronologically 3. For each extracted entity pair: - Creates/updates Entity node with 768-dim embedding - Creates/updates Concept node with 768-dim embedding - Links both to the Event via `[:EXTRACTED_FROM]` - Creates the domain relationship (e.g., `[:IS_A_PATTERN_OF]`)

After this call, the full Knowledge Graph looks like:

```
(Interaction: session_82dd0f7b)
    ├── Message: user input
    ├── Message: agent response
    └── ← [:SUMMARIZES] ── (Event: "User asked about Clean Architecture...")
                                ├── ← [:EXTRACTED_FROM] ── (Entity: "Clean A
                                │                               └── [:IS_A_PA
                                └── ← [:EXTRACTED_FROM] ── (Entity: "Neo4j D
                                                                └── [:BELONGS
```

Phase 2+3: Retrieval Engine — Semantic Recall

```
    print("\n==================================")
    print("[Retrieval Engine] Testing Spreading Activation...")
    retriever = RetrievalEngine(cutoff_threshold=0.2, max_depth=3)

    # Search for an anchor using a synonymous term to inherently force Seman
    query = ["System Design"]
    print(f"Triggering Search for Concept: {query}")

    results = retriever.retrieve(query)
```

The deliberate test: We search for `"System Design"` — a term that does NOT exist in the graph. This forces the system through the full dual-trigger pipeline: 1. Lexical search → MISS (no node named "System Design") 2. Semantic vector fallback → HIT ("System Design" ≈ "Software Design" at cosine 0.95) 3. Spreading Activation → BFS fan-out from anchors 4. Lateral Inhibition → Filter out hubs

```
    if results["status"] == "Hit":
        print(f"\n[ANCHORS FOUND]: {results['anchors']}")
        print(f"[ACTIVATED NETWORK]:")
        for node in results["network"]:
            print(f"  -> [{node['distance']} hops away] {node['type']}: {nod
        print(f"[INHIBITED HUBS]: {results['inhibited_hubs']}")
    else:
        print("\n[MISS] No relevant anchors found.")
```

Expected output:

```
[ANCHORS FOUND]: ['Software Design', 'Clean Architecture']
[ACTIVATED NETWORK]:
  -> [0 hops away] Entity: Software Design
  -> [0 hops away] Entity: Clean Architecture
  -> [2 hops away] Event: None
  -> [2 hops away] Concept: Software Design
  -> [3 hops away] Entity: Neo4j Driver
  -> [3 hops away] Concept: Infrastructure
  ...
```

```
    print("\nPhase 3 Vector Semantic Verification Complete! Check /Logs for

if __name__ == "__main__":
    sys.exit(main())
```

`sys.exit(main())` — `main()` returns `None` (implicitly), which maps to exit code 0 (success). If an unhandled exception occurs, Python exits with code 1.

How to Use This as a Template

To add GraphCortex to your own AI agent:

```
from graph_cortex.core.memory.manager import MemoryManager
from graph_cortex.core.retrieval.engine import RetrievalEngine

# 1. Ingest a conversation
manager = MemoryManager()
manager.process_turn(session_id="chat_001", user_input="...", agent_response

# 2. Consolidate with LLM-extracted facts
manager.consolidate_episode("chat_001", summary="...", extracted_entities=[.
```

```
# 3. Recall relevant knowledge
engine = RetrievalEngine()
results = engine.retrieve(["relevant search terms"])

# 4. Feed the activated network into your LLM's context
context = results["network"]  # Pass this to your prompt
```

Module 15: End-to-End Data Flow Walkthrough

The Complete Journey of a Single Conversation

This module traces the **exact path** that data takes through the entire GraphCortex system, from the moment a user sends a message to the moment the AI recalls it using a synonym.

Act 1: The Conversation Happens

User says: *"How does clean architecture improve graph database access?"*
Agent says: *"By strictly decoupling the Neo4j driver queries (infrastructure) from the raw data structures (core domain), the codebase becomes modular and easily testable."*

Call Chain:

```
main.py
  └→ MemoryManager.process_turn()
       ├→ WorkingMemory.add_interaction("session_82dd")
       │    └→ Neo4j: MERGE (i:Interaction {session_id: "session_82dd"})
       │
       ├→ WorkingMemory.add_message("session_82dd", "user", "How does...")
       │    └→ Neo4j: MATCH (i) → CREATE (m:Message) → CREATE (i)-[:CONTAINS
       │              No previous message → FOREACH skips [:NEXT] creation
       │
       └→ WorkingMemory.add_message("session_82dd", "agent", "By strictly...
```

```
            └→ Neo4j: MATCH (i) → CREATE (m2:Message) → CREATE (i)-[:CONTAIN
                      Found previous message m1 → FOREACH creates (m1)-[:NEX
```

Neo4j State After Act 1:

```
┌──────────────────────────────────────────────┐
│             WORKING MEMORY                   │
│                                              │
│  (Interaction: session_82dd)                 │
│       │                                      │
│       ├──[:CONTAINS]──→ (Message: user)      │
│       │                    │                 │
│       │                    └── [:NEXT] ──→   │
│       │                                      │
│       └──[:CONTAINS]──→ (Message: agent)     │
│                                              │
│  Total: 3 nodes, 4 relationships             │
└──────────────────────────────────────────────┘
```

Act 2: The Episode Gets Consolidated

An LLM generates a summary and extracts structured facts.

Call Chain:

```
main.py
  └→ MemoryManager.consolidate_episode()
       │
       ├→ PropertySharder.store("ep_session_82dd", "User asked about...")
       │    └→ Mode: local → returns the string unchanged
       │
       ├→ EpisodicMemory.create_event("session_82dd", "User asked about...")
       │    └→ Neo4j: MATCH (i:Interaction) → CREATE (e:Event)
```

```
        │              CREATE (e)-[:SUMMARIZES]->(i)
        │              No previous event → FOREACH skips [:FOLLOWS]
        │              RETURN event_id = "45a3e13b-..."
        │
        ├→ SemanticMemory.add_entity("Clean Architecture")
        │    ├→ _get_embedding("Clean Architecture")
        │    │    └→ SentenceTransformer('bge-base-en-v1.5', device='mps')
        │    │       └→ encode() → [0.031, -0.089, 0.124, ...] (768 floats)
        │    └→ Neo4j: MERGE (e:Entity {name: "Clean Architecture"})
        │              SET e.embedding = [0.031, -0.089, ...]
        │
        ├→ SemanticMemory.add_entity("Software Design")
        │    ├→ _get_embedding("Software Design")
        │    │    └→ (model already loaded — cache hit)
        │    │       └→ encode() → [0.042, -0.076, 0.118, ...] (768 floats)
        │    └→ Neo4j: MERGE (e:Entity {name: "Software Design"})
        │              SET e.embedding = [0.042, -0.076, ...]
        │
        └→ SemanticMemory.extract_from_event(event_id, "Clean Architecture",
             └→ Neo4j: MATCH (ev:Event {event_id: "45a3e13b-..."})
                       MERGE (e:Entity {name: "Clean Architecture"})
                       SET e.embedding = $entity_vector
                       MERGE (c:Concept {name: "Software Design"})
                       SET c.embedding = $concept_vector
                       MERGE (e)-[:EXTRACTED_FROM]->(ev)
                       MERGE (c)-[:EXTRACTED_FROM]->(ev)
                       MERGE (e)-[:IS_A_PATTERN_OF]->(c)
```

Neo4j State After Act 2:

```
┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  ┌─ WORKING MEMORY ──────────┐     ┌─ EPISODIC MEMORY ──────────┐   │
│  │ (Interaction: session_82dd)│     │ (Event: "User asked about  │   │
│  │    ├── Message: user       │←────│  Clean Architecture...")   │   │
│  │    └── Message: agent      │  [:SUMMARIZES]                   │   │
│  └────────────────────────────┘     └──────────────┬─────────────┘   │
│                                                     │                 │
│                                   [:EXTRACTED_FROM]                  │
```

```
│                                                  │                    │
│  ┌─ SEMANTIC MEMORY ──────────────────────────────┼──────────────────┐   │
│  │                                                │                  │   │
│  │  (Entity: "Clean Architecture")  ←─────────────┘                  │   │
│  │     embedding: [0.031, -0.089, ...]                               │   │
│  │         │                                                         │   │
│  │         └── [:IS_A_PATTERN_OF] ──→ (Concept: "Software Design")│   │
│  │                                     embedding: [0.042, -0.076] │   │
│  │                                                                   │   │
│  │  (Entity: "Neo4j Driver")                                        │   │
│  │     embedding: [0.055, -0.023, ...]                               │   │
│  │         │                                                         │   │
│  │         └── [:BELONGS_TO_LAYER] ──→ (Concept: "Infrastructure")│   │
│  │                                     embedding: [-0.011, 0.089] │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
│  Total: 9 nodes, 11 relationships, 4 vector embeddings                  │
└──────────────────────────────────────────────────────────────────────────┘
```

Act 3: The AI Recalls Using a Synonym

User searches for: *"System Design"* (this exact string does NOT exist in the graph)

Call Chain:

```
main.py
  └→ RetrievalEngine.retrieve(["System Design"])
       │
       ├→ STEP 1A: get_anchor_nodes_by_name(session, ["System Design"])
       │     └→ Neo4j: MATCH (n) WHERE n.name CONTAINS "System Design"
       │        Result: [] ← LEXICAL MISS
       │
       ├→ Logger: "Lexical Miss for '['System Design']'"
       │
       ├→ STEP 1B: Semantic Vector Fallback
```

```
│    ├→ SentenceTransformer.encode("System Design")
│    │    └→ MPS GPU: [0.039, -0.082, 0.121, ...] (768 floats)
│    │
│    └→ get_anchors_by_vector_similarity(session, vector, limit=2)
│         └→ Neo4j: CALL db.index.vector.queryNodes('entity_vector_in
│            HNSW search across all Entity embeddings
│
│            Cosine("System Design", "Software Design")  = 0.9484 ✅ >
│            Cosine("System Design", "Clean Architecture") = 0.8224 ✅
│            Cosine("System Design", "Neo4j Driver")     = 0.3102 ❌ <
│            Cosine("System Design", "Infrastructure")   = 0.4891 ❌ <
│
│            Result: [{"name": "Software Design", "score": 0.9484},
│                     {"name": "Clean Architecture", "score": 0.8224}
│
├→ Logger: "Semantic Fallback Success! Found semantic anchors: [...]"
│
├→ STEP 2: Spreading Activation (for each anchor)
│    │
│    ├→ Anchor: "Software Design" (AE = 1.0)
│    │    └→ execute_spreading_activation_hop(session, node_id, depth
│    │         └→ Neo4j: MATCH path = (start)-[*1..3]-(connected)
│    │            Returns: Event_001 (dist=2, deg=5), Interaction (di
│    │
│    └→ Anchor: "Clean Architecture" (AE = 1.0)
│         └→ execute_spreading_activation_hop(session, node_id, depth
│              └→ Neo4j: MATCH path = (start)-[*1..3]-(connected)
│                 Returns: Software Design (dist=2, deg=3), Event_001
│
├→ STEP 3: Lateral Inhibition (for each anchor's network)
│    └→ apply_lateral_inhibition(traversed_nodes, cutoff_threshold=0.
│         For each node:
│           AE = 1.0 / (((dist × 0.5) + 1) × ((deg × 0.1) + 1))
│           Keep if AE >= 0.2, otherwise inhibit
│
└→ STEP 4: Deduplicate overlapping networks
     └→ Merge by node_id, keep highest AE
        Return final activated knowledge sub-graph
```

The Final Output:

```
{
    "status": "Hit",
    "anchors": ["Software Design", "Clean Architecture"],
    "network": [
        {"name": "Software Design",    "type": "Entity",      "distance": 0,
        {"name": "Clean Architecture",  "type": "Entity",      "distance": 0
        {"name": "Software Design",     "type": "Concept",     "distance": 2
        {"name": "Neo4j Driver",        "type": "Entity",      "distance": 3
        {"name": "Infrastructure",      "type": "Concept",     "distance": 3
        ...
    ],
    "inhibited_hubs": [...]
}
```

The Key Insight

The user searched for **"System Design"**. This string appears NOWHERE in the database. Yet the system:

1. Recognised that "System Design" is **semantically similar** to "Software Design" (0.95 cosine)
2. Also found "Clean Architecture" as a related anchor (0.82 cosine)
3. Traversed outward to discover the full connected knowledge network
4. Filtered out irrelevant hubs
5. Returned a rich, structured sub-graph the AI can reason over

This is the difference between "searching for text" and "understanding meaning."

Summary: The Full Technology Stack

```
┌──────────────────────────────────────────────────────────┐
│                      USER INPUT                          │
│                  "System Design"                         │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│               BAAI/bge-base-en-v1.5                      │
│            (109M params, 768-dim output)                 │
│              Apple Silicon MPS GPU                       │
└──────────────────────────┬───────────────────────────────┘
                           │ 768-dim vector
                           ▼
┌──────────────────────────────────────────────────────────┐
│              Neo4j 5.x Vector Index                      │
│            HNSW + Cosine Similarity                      │
│        entity_vector_index (768-dim)                     │
└──────────────────────────┬───────────────────────────────┘
                           │ Anchor nodes
                           ▼
┌──────────────────────────────────────────────────────────┐
│          Neo4j Cypher BFS Traversal                      │
│         Variable-length paths [*1..3]                    │
│         Returns distance + degree                        │
└──────────────────────────┬───────────────────────────────┘
                           │ Raw network
                           ▼
┌──────────────────────────────────────────────────────────┐
│           Lateral Inhibition (Python)                    │
│    AE = E / ((dist_factor) × (degree_factor))           │
│         cutoff_threshold = 0.2                           │
└──────────────────────────┬───────────────────────────────┘
                           │ Filtered network
                           ▼
┌──────────────────────────────────────────────────────────┐
│            Activated Knowledge Sub-Graph                  │
│                                                          │
```

```
│   Ready to be injected into an LLM's context window     │
└──────────────────────────────────────────────────────────┘
```

📋 Phase 1 Report Card — Storage Substrate & Multi-Layer Memory Framework

---

Project: GraphCortex — Distributed Neuro-Symbolic Memory Grid
Phase: 1 — Storage Substrate (MLMF)
Date: 12–13 April 2026
Status: ✅ **COMPLETE**

🎯 Objective

Build the foundational storage substrate for a neuro-symbolic memory system. Replace flat vector databases with a structured, multi-layered Neo4j Knowledge Graph capable of storing Working Memory (real-time conversations), Episodic Memory (chronological event summaries), and Semantic Memory (entity-level abstractions) — all interconnected in a single graph topology.

📊 Final Scorecard

Component	Plan	Delivered	Grade
Docker Infrastructure	Neo4j container with APOC + GDS	`docker-compose.yml` with Neo4j 5.18.1, APOC, GDS	**A**

Component	Plan	Delivered	Grade
Environment Config	`.env` template for connection vars	`.env` + `.env.example` with URI/user/password	**A**
Database Connectivity	Thread-safe Neo4j driver	Singleton `Neo4jConnection` class with `get_session()`	**A**
Schema Migrations	Constraints + indexes	Uniqueness constraints + timestamp indexes for all layers	**A**
Working Memory	`Interaction` + `Message` nodes	Delivered with `[:CONTAINS]` and `[:NEXT]` chaining	**A**
Episodic Memory	`Event` nodes with chronological chains	Delivered with `[:SUMMARIZES]` and `[:FOLLOWS]` relationships	**A**
Semantic Memory	`Entity` + `Concept` extraction	Delivered with `[:RELATED_TO]` and `[:EXTRACTED_FROM]` links	**A**
Memory Manager	Orchestrator pipeline	`MemoryManager` with `process_turn()` + `consolidate_episode()`	**A**
Property Sharding	Abstraction for heavy payload offloading	`PropertySharder` with local/S3 modes	**A**
Clean Architecture	Domain-driven folder structure	`core/` → `infrastructure/` → `interfaces/` separation	**A+**

> ***Overall Grade: A***

📦 Files Created

Infrastructure Layer

File	Purpose
docker-compose.yml	Neo4j 5.18.1 container with APOC + GDS plugins, port mapping (7474/7687), health checks
.env	`NEO4J_URI`, `NEO4J_USERNAME`, `NEO4J_PASSWORD` environment variables
neo4j_connection.py	Singleton Neo4j driver with `get_session()` and `execute_read_query()`
schema_migrations.py	Constraint + index initialization for all memory layers
sharding.py	Property sharding abstraction (local passthrough / S3 URI mode)

Core Memory Layer

File	Purpose
working.py	`WorkingMemory` class — `add_interaction()`, `add_message()` with `[:NEXT]` chaining
episodic.py	`EpisodicMemory` class — `create_event()` with `[:FOLLOWS]` chronological chains
semantic.py	`SemanticMemory` class — `add_entity()`, `extract_from_event()` with relational links
manager.py	`MemoryManager` orchestrator — `process_turn()` + `consolidate_episode()` pipeline

Project Foundation

File	Purpose
pyproject.toml	Project metadata, dependencies (`neo4j>=5.0.0` , `python-dotenv>=1.0.0`), CLI entry point
README.md	Full project documentation with architecture overview and quickstart
DECISIONS.md	Architectural decision records (Neo4j vs flat vectors, Clean Architecture rationale)

🏗️ Architecture Established

```
graph TD
    subgraph "Clean Architecture"
        direction TB

        subgraph "Core - The Brain"
            WM["WorkingMemory"]
            EM["EpisodicMemory"]
            SM["SemanticMemory"]
            MM["MemoryManager"]
            MM --> WM
            MM --> EM
            MM --> SM
        end

        subgraph "Infrastructure - The Tools"
            DB["Neo4jConnection - Singleton"]
            SCH["Schema Migrations"]
            SH["PropertySharder"]
        end

        subgraph "Interfaces - The Delivery"
            CLI["CLI main.py"]
        end
```

```
        CLI --> MM
        MM --> SH
        WM --> DB
        EM --> DB
        SM --> DB
        SCH --> DB
    end
```

🗃️ Neo4j Graph Schema

```
graph LR
    subgraph "Working Memory"
        I["Interaction"] -->|CONTAINS| M1["Message"]
        M1 -->|NEXT| M2["Message"]
    end

    subgraph "Episodic Memory"
        E1["Event"] -->|FOLLOWS| E2["Event"]
        E1 -->|SUMMARIZES| I
    end

    subgraph "Semantic Memory"
        EN["Entity"] -->|EXTRACTED_FROM| E1
        C["Concept"] -->|EXTRACTED_FROM| E1
        EN -->|RELATED_TO| C
    end
```

Constraints & Indexes Created

Type	Target	Property
Index	`Interaction`	`timestamp`

Type	Target	Property
Index	`Message`	`message_id`
Unique Constraint	`Event`	`event_id`
Index	`Event`	`timestamp`
Unique Constraint	`Entity`	`name`
Unique Constraint	`Concept`	`name`

📜 Git History

Date	Commit	Description
12 Apr	`d435b0d`	Initial commit
13 Apr	`36e6586`	Restructure to domain-driven design + Phase 1 MLMF
13 Apr	`d4d5344`	Update README with architecture and roadmap
13 Apr	`078c9a0`	Move implementation plan to `docs/` directory
14 Apr	`4bf94a4`	Polish episodic queries + clarify singleton docs

📝 Summary

Phase 1 established the entire foundation of GraphCortex — from the Docker containerisation and Neo4j driver connectivity to the full Multi-Layer Memory Framework. The three-tier memory system (Working → Episodic → Semantic) was implemented with

strict Clean Architecture separation, ensuring the core mathematical memory logic has absolutely zero dependencies on the database layer. The `MemoryManager` orchestrator provides a clean API (`process_turn()` + `consolidate_episode()`) that downstream phases build upon. Property sharding was abstracted early to allow future migration to distributed storage (S3/MongoDB) without touching core logic.

📋 Phase 2 Report Card — Spreading Activation Retrieval Engine

Project: GraphCortex — Distributed Neuro-Symbolic Memory Grid
Phase: 2 — Retrieval Engine
Date: 14 April 2026
Status: ✅ **COMPLETE**

🎯 Objective

Build a brain-inspired retrieval engine that replaces standard cosine similarity with a dynamic Spreading Activation algorithm. The engine fans outward from anchor nodes, mathematically attenuating activation energy via the Fan Effect (degree penalty) and distance decay, while applying Lateral Inhibition to suppress generic hub nodes that would otherwise flood the AI's context window.

📊 Final Scorecard

Component	Plan	Delivered	Grade
Dual-Trigger Initialization	Lexical + Semantic trigger routing	Lexical trigger active, Semantic stub prepared for Phase 3	**A**
RetrievalEngine Class	Core orchestrator	`RetrievalEngine` with configurable `cutoff_threshold` + `max_depth`	**A**

Component	Plan	Delivered	Grade
	for recall		
Spreading Activation (BFS)	Custom Cypher traversal up to depth 3	Parameterised BFS query returning distance + degree metrics	**A**
Energy Decay Formula	Mathematical fan-effect attenuation	`AE = E / (((d × dp) + 1) × ((deg × degp) + 1))` implemented	**A**
Lateral Inhibition	Filter overly generic hubs	`apply_lateral_inhibition()` with configurable cutoff threshold	**A**
Infrastructure Queries	Scalable, parameterised Cypher	`get_anchor_nodes_by_name()` + `execute_spreading_activation_hop()`	**A**
CLI Verification	End-to-end retrieval demo	Updated `main.py` with full ingestion → retrieval pipeline	**A**
Clean Architecture	Core/Infra separation	Math in `core/retrieval/`, Cypher in `infrastructure/db/queries/`	**A+**

> ***Overall Grade: A***

📂 Files Created & Modified

Core Retrieval Logic (The Math)

File	Purpose
engine.py	`RetrievalEngine` class — Dual-Trigger routing, energy decay orchestration, network deduplication
inhibition.py	Lateral Inhibition module — Fan Effect energy decay formula, hub suppression

Infrastructure Queries (The Execution)

File	Purpose
retrieval_queries.py	`get_anchor_nodes_by_name()` (Lexical) + `execute_spreading_activation_hop()` (BFS)
neo4j_connection.py	Added `execute_read_query()` for safe read-only transactions

Interface

File	Purpose
main.py	Updated CLI with full ingestion → retrieval verification pipeline

---

□ The Energy Decay Formula

The core mathematical innovation of Phase 2 is the Activation Energy (AE) decay function, designed to simulate how the biological brain attenuates signal strength as it fans outward through neural networks:

```
AE = initial_energy / (((distance × distance_penalty) + 1) × ((degree × degr
```

Parameters

Parameter	Default	Purpose
`initial_energy`	`1.0`	Maximum energy assigned to exact anchor matches
`distance_penalty`	`0.5`	How fast energy decays with each hop away from the anchor
`degree_penalty`	`0.1`	How fast energy decays for highly connected (generic) hub nodes
`cutoff_threshold`	`0.2`	Minimum AE required to survive — nodes below this are inhibited

Worked Example

```
Node: "Neo4j Driver" (3 hops away, degree 4)
AE = 1.0 / (((3 × 0.5) + 1) × ((4 × 0.1) + 1))
AE = 1.0 / (2.5 × 1.4)
AE = 1.0 / 3.5
AE = 0.2857 ✅ (Above 0.2 threshold — SURVIVES)

Node: "Generic Hub" (3 hops away, degree 20)
AE = 1.0 / (((3 × 0.5) + 1) × ((20 × 0.1) + 1))
AE = 1.0 / (2.5 × 3.0)
AE = 1.0 / 7.5
AE = 0.1333 ❌ (Below 0.2 threshold — INHIBITED)
```

🏗️ Retrieval Pipeline Architecture

```
graph TD
    A["User Query"] --> B{"Lexical Trigger"}
    B -->|"Exact Match"| D["Anchor Nodes"]
    B -->|"No Match"| C["Semantic Stub - Phase 3"]
```

```
    C -.->|"Placeholder"| D
    D --> E["Spreading Activation - BFS depth 3"]
    E --> F["Raw Traversed Network"]
    F --> G["Lateral Inhibition"]
    G --> H{"AE >= cutoff?"}
    H -->|"Yes"| I["Activated Network"]
    H -->|"No"| J["Inhibited Hubs - Dropped"]
    I --> K["Deduplicated Result"]

    style C fill:#333,stroke:#666,color:#999
    style J fill:#8b0000,stroke:#cc0000,color:#fff
    style K fill:#006400,stroke:#00cc00,color:#fff
```

> *[!NOTE] The Semantic Trigger (dashed box) was deliberately left as a stub in Phase 2. It was fully activated in Phase 3 with the `bge-base-en-v1.5` vector fallback.*

🔧 Key Design Decisions

1. Clean Architecture Separation

The retrieval engine is split into two strict layers: - **`core/retrieval/`** — Contains only pure mathematical functions (`inhibition.py`) and orchestration logic (`engine.py`). Zero database imports. - **`infrastructure/db/queries/`** — Contains raw Cypher queries that return distance/degree metrics. The core layer consumes these metrics without knowing they came from Neo4j.

2. Configurable Thresholds

All decay parameters (`cutoff_threshold` , `max_depth` , `degree_penalty` , `distance_penalty`) are configurable at `RetrievalEngine` instantiation, allowing fine-tuning per use case.

3. Network Deduplication

When multiple anchors are activated, their spreading activation networks can overlap. The engine deduplicates by `node_id`, always keeping the instance with the highest activation energy.

📜 Git History

Date	Commit	Description
14 Apr	`950d213`	Implement dual-trigger spreading activation retrieval engine with lateral inhibition
14 Apr	`6f81cd7`	Rename implementation_plan to implementation_plan_phase1 for clarity

---

📝 Summary

Phase 2 delivered the cognitive retrieval engine at the heart of GraphCortex. Unlike standard RAG systems that return flat, isolated document chunks, the Spreading Activation algorithm traverses the Knowledge Graph outward from anchor nodes, building a connected associative sub-graph that the AI can reason over. The Lateral Inhibition module mathematically prevents the "Hub Explosion" problem — where overly generic concepts (like "User" or "Project") would otherwise dominate the AI's context window. The architecture was designed with a deliberate Semantic Trigger stub, cleanly bridged in Phase 3 with vector embeddings.

📋 Phase 3 Report Card — Semantic Vector Integration

Project: GraphCortex — Distributed Neuro-Symbolic Memory Grid
Phase: 3 — Semantic Vector Integration
Date: 16 April 2026
Status: ✅ **COMPLETE**

🎯 Objective

Bridge the gap between text-based matching and true semantic understanding by embedding Knowledge Graph nodes into a high-dimensional vector space inside Neo4j. This enables the AI to recall concepts even when completely different words are used (e.g., searching for `"System Design"` retrieves `"Clean Architecture"` and `"Software Design"`).

📊 Final Scorecard

Component	Plan	Delivered	Grade
Neo4j Vector Indexes	384-dim indexes	768-dim indexes (upgraded)	**A+**
Embedding Model	`all-MiniLM-L6-v2`	`BAAI/bge-base-en-v1.5` (upgraded)	**A+**

Component	Plan	Delivered	Grade
Hardware Acceleration	Not planned	Apple Silicon MPS GPU (`device='mps'`)	**A+**
Vector Similarity Query	`get_anchors_by_vector_similarity()`	Delivered + Neo4j 5.x compat fixes	**A**
Ingestion Pipeline	Embed on `add_entity` + `extract_from_event`	Delivered with lazy-loading	**A**
Retrieval Engine Fallback	Lexical → Semantic fallback	Fully wired dual-trigger pipeline	**A**
Logging System	Timestamped `/Logs` directory	Delivered — 5 verified log files	**A**
CLI Verification Script	Demo synonym recall	Delivered — "System Design" → "Clean Architecture"	**A**
APOC Migration	Not planned	Migrated `apoc.do.when` → native Cypher	**A+**

> ***Overall Grade: A+***

📂 Files Modified

Core Memory Layer

File	Change
semantic.py	Upgraded to `bge-base-en-v1.5` with `device='mps'`, 768-dim embeddings on ingestion
working.py	Migrated `apoc.do.when` → native Cypher `FOREACH` conditional
episodic.py	Migrated `apoc.do.when` → native Cypher `FOREACH` conditional

Retrieval Layer

File	Change
engine.py	Activated semantic vector fallback, upgraded to `bge-base-en-v1.5` + MPS
retrieval_queries.py	Added `get_anchors_by_vector_similarity()`, fixed `$depth` param + `SIZE()` for Neo4j 5.x

Infrastructure

File	Change
schema_migrations.py	`VECTOR_DIMENSION` 384→768, `similarity.function` → `vector.similarity_function`, added DROP+CREATE
.env	Updated `NEO4J_URI` to `neo4j://127.0.0.1:7687`

Configuration & Logging

File	Change
logger.py	New — timestamped retrieval logger writing to `/Logs`
main.py	Updated CLI to demo synonym-based semantic retrieval

🔬 Model Upgrade Details

```
┌──────────────────────────┬──────────────────────┬──────────────────────
│                          │ Before (Phase 3 v1)  │ After (Phase 3 Final)
├──────────────────────────┼──────────────────────┼──────────────────────
│ Model                    │ all-MiniLM-L6-v2     │ BAAI/bge-base-en-v1.5
│ Parameters               │ ~22 Million          │ ~109 Million
│ Vector Dimensions        │ 384                  │ 768
│ MTEB Ranking             │ Mid-tier             │ Top-tier (open-source
│ Hardware                 │ CPU only             │ Apple Silicon MPS GPU
│ Inference Device         │ torch.device('cpu')  │ torch.device('mps')
└──────────────────────────┴──────────────────────┴──────────────────────
```

🐛 Neo4j 5.x Compatibility Issues Encountered & Resolved

During integration with Neo4j Desktop (v5.x), four compatibility issues were discovered and patched:

Issue 1: Vector Index Config Key

```
- OPTIONS {indexConfig: {`similarity.function`: 'cosine'}}
```

```
+ OPTIONS {indexConfig: {`vector.similarity_function`: 'cosine'}}
```

> *Neo4j 5.x renamed the config key. Patched in `schema_migrations.py`.*

Issue 2: Parameterised Variable-Length Patterns

```
- MATCH path = (start)-[*1..$depth]-(connected)
+ MATCH path = (start)-[*1..{depth}]-(connected)  // f-string with int() san
```

> *Neo4j 5.x disallows `$parameters` inside `[*1..$depth]`. Patched in `retrieval_queries.py`.*

Issue 3: Pattern Expressions in Non-Boolean Context

```
- SIZE((connected)--()) AS degree
+ COUNT { (connected)--() } AS degree
```

> *Neo4j 5.x restricts bare pattern expressions to boolean contexts. Patched in `retrieval_queries.py`.*

Issue 4: Deprecated APOC Procedures

```
- CALL apoc.do.when(last IS NOT NULL, 'CREATE (last)-[:NEXT]->(m) ...', ...)
+ FOREACH (_ IN CASE WHEN last IS NOT NULL THEN [1] ELSE [] END |
+     CREATE (last)-[:NEXT]->(m)
+ )
```

> *`apoc.do.when` deprecated in Neo4j 5.x. Migrated to native Cypher in `working.py` and `episodic.py`.*

✅ Verification Results

Semantic Retrieval Test

- **Query:** `"System Design"` (not present in the Knowledge Graph)
- **Lexical Trigger:** ✕ Miss (expected — no exact string match)
- **Semantic Vector Fallback:** ✅ Hit

Cosine Similarity Scores (from retrieval log)

Anchor Found	Cosine Score	Verdict
`Software Design`	**0.9484**	🟢 Near-perfect semantic match
`Clean Architecture`	**0.8224**	🟢 Strong conceptual association

> *[!NOTE] Both scores exceeded the `0.65` minimum similarity threshold configured in `retrieval_queries.py`, confirming the 768-dimensional `bge-base` model delivers significantly higher semantic resolution than the previous 384-dimensional `MiniLM` model.*

Spreading Activation Output

The full activated network from the semantic anchors:

```
→ [0 hops] Entity: Software Design
→ [0 hops] Entity: Clean Architecture
→ [2 hops] Event (episodic links)
→ [2 hops] Concept: Software Design
→ [3 hops] Entity: Neo4j Driver
→ [3 hops] Concept: Infrastructure
→ [3 hops] Interaction (working memory)
→ [4 hops] Message (conversation turns)
```

Log Files Generated

5 retrieval logs successfully written to `/Logs`:

```
Logs/retrieval_20260416_184741.log
Logs/retrieval_20260416_184805.log
Logs/retrieval_20260416_184907.log
Logs/retrieval_20260416_184937.log
Logs/retrieval_20260416_195809.log
```

🏗️ Architecture After Phase 3

```
graph TD
    A["User Query: 'System Design'"] --> B{"Lexical Trigger"}
    B -->|Hit| D["Anchor Nodes"]
    B -->|Miss| C{"Semantic Vector Fallback"}
    C --> E["bge-base-en-v1.5 on MPS GPU"]
    E --> F["768-dim Embedding"]
    F --> G["Neo4j Vector Index - Cosine"]
    G --> D
    D --> H["Spreading Activation - BFS"]
    H --> I["Lateral Inhibition - Fan Decay"]
    I --> J["Activated Knowledge Network"]
```

```
    style A fill:#1a1a2e,stroke:#e94560,color:#fff
    style C fill:#16213e,stroke:#0f3460,color:#fff
    style E fill:#0f3460,stroke:#53354a,color:#fff
    style J fill:#533483,stroke:#e94560,color:#fff
```

📝 Summary

Phase 3 successfully transformed GraphCortex from a purely lexical retrieval system into a **hybrid semantic-lexical engine**. The system now understands meaning, not just strings. A query for `"System Design"` correctly recalls `"Software Design"` (0.95 cosine) and `"Clean Architecture"` (0.82 cosine) — concepts that share zero common words but deep semantic relationships.

The upgrade to `bge-base-en-v1.5` with Apple Silicon MPS acceleration ensures this runs at production speed on the M4 MacBook Air, and the migration away from APOC dependencies future-proofs the codebase against Neo4j version changes.